

**CONCEITOS-
-CHAVE**

arquétipos	241
arquitetura	230
alternativas	245
centralizada em dados	235
complexidade	247
componentes	243
em camadas	238
estilos	234
fluxo de dados	236
gêneros	232
orientada a objetos	230
padrões	238
projeto	239
refinamento	242
templates	233
ATAM	245
fabricação	268
instância	243
linguagem de descrição de arquitetura	247
mapeamento	248

Projeto foi descrito como um processo em várias etapas em que as representações de dados e a estrutura do programa, as características de interfaces e os detalhes procedurais são sintetizados com base nos requisitos de informação. Essa descrição é estendida por Freeman [Fre80]:

[P]rojeto é uma atividade que trata da tomada de decisões importantes, frequentemente de natureza estrutural. Ele divide com a programação a responsabilidade em abstrair a representação de informação e de sequências de processamento, porém o nível de detalhes é bastante diverso nos extremos. Um projeto constrói representações de programas coerentes e bem planejados que se concentram nas inter-relações das partes em um alto nível e nas operações lógicas envolvidas em níveis mais baixos.

Conforme citado no Capítulo 8, o projeto é dirigido por informações. Os métodos de projeto de software são obtidos considerando-se cada um dos três domínios do modelo de análise. Os domínios de dados, funcional e comportamental servem de orientação para a criação do projeto de software.

Neste capítulo serão apresentados os métodos necessários para criar “representações coerentes e bem planejadas” das camadas de dados e da arquitetura do modelo de projeto. O objetivo é fornecer uma abordagem sistemática para a obtenção do projeto da arquitetura — o esquema preliminar por meio do qual o software é construído.

PANORAMA

O que é? O projeto da arquitetura representa a estrutura de dados e os componentes de programa necessários para construir um sistema computacional. Ele considera o estilo de arquitetura que o sistema assumirá, a estrutura e as propriedades dos componentes que constituem o sistema, bem como as inter-relações que ocorrem entre todos os componentes da arquitetura de um sistema.

Quem realiza? Embora um engenheiro de software possa projetar tanto os dados quanto a arquitetura, essa tarefa frequentemente é distribuída a especialistas ao se construírem sistemas grandes e complexos. Um projetista de bancos de dados ou de depósito de dados cria a arquitetura de dados para um sistema. O “arquiteto de sistemas” escolhe um estilo de arquitetura apropriado com base nos requisitos obtidos durante a análise de requisitos.

Por que é importante? Você não tentaria construir uma casa sem uma planta, não é mesmo? Você também não desenharia as plantas começando pela distribuição dos encanamentos da casa. Deve-se partir do contexto geral — a casa em

si — antes de se preocupar com os detalhes. É exatamente isso o que faz o projeto da arquitetura — ele dá uma visão geral e garante que você o entendeu de forma correta.

Quais são as etapas envolvidas? O projeto da arquitetura começa pelo projeto de dados e então prossegue para a derivação de uma ou mais representações da estrutura da arquitetura do sistema. São analisados estilos ou padrões de arquitetura alternativos para obter uma estrutura mais adequada aos requisitos do cliente e atributos de qualidade. Uma vez que tenha se escolhido uma alternativa, a arquitetura é elaborada usando-se um método de projeto da arquitetura.

Qual é o artefato? Durante o projeto da arquitetura é criado um modelo de arquitetura que engloba a arquitetura de dados e a estrutura de programas. Além disso, são descritas as propriedades e as relações (interações) entre os componentes.

Como garantir que o trabalho foi realizado corretamente? A cada estágio, são revisados os produtos resultantes do projeto de software em termos de clareza, correção, completude e consistência com os requisitos e entre si.

9.1 ARQUITETURA DE SOFTWARE

Em seu famoso livro sobre o tema, Shaw e Garlan [Sha96] discutem a arquitetura de software da seguinte maneira:

Desde que o primeiro programa foi dividido em módulos, os sistemas de software passaram a ter arquiteturas e os programadores passaram a ser responsáveis pelas interações entre os módulos e as propriedades globais do conjunto. Historicamente, as arquiteturas têm sido implícitas — acidentes de implementação ou sistemas legados do passado. Os bons desenvolvedores de software muitas vezes adotaram um ou vários padrões de arquitetura como estratégias para organização de sistemas, porém usam tais padrões informalmente e não possuem nenhum meio para torná-los explícitos no sistema resultante.

Hoje, arquitetura de software efetiva e sua representação explícita tornaram-se temas dominantes em engenharia de software.

9.1.1 O que é arquitetura?

Ao se considerar a arquitetura de um edifício, vários atributos diferentes vêm à mente. No nível mais simplista, pensamos na forma geral da estrutura física. Mas, na realidade, arquitetura é muito mais do que isso. Ela é a maneira pela qual os vários componentes do edifício são integrados para formar um todo coeso. É o modo pelo qual o edifício se ajusta em seu ambiente e integra com outros edifícios da vizinhança. É o grau com que o edifício atende seu propósito expresso e satisfaz às necessidades de seu proprietário. É o sentido estético da estrutura — o impacto visual do edifício — e a maneira pela qual as texturas, cores e materiais são combinados para criar a fachada e o “ambiente de moradia”. Ela engloba também os detalhes — o projeto dos dispositivos de iluminação, o tipo de piso, o posicionamento de painéis, enfim, a lista é interminável. E, finalmente, ela é arte. Mas arquitetura também é algo mais. É constituída por “milhares de decisões, tanto as grandes como as pequenas” [Tyr05]. Algumas dessas decisões são tomadas logo no início do projeto e podem ter um impacto profundo sobre todas as ações subsequentes. Outras são postergadas ao máximo, eliminando, portanto, restrições demasiadas que levariam a uma implementação inadequada do estilo da arquitetura. Mas o que dizer da arquitetura de software? Bass, Clements e Kazman [Bas03] definem esse termo difícil de descrever da seguinte maneira:

A arquitetura de software de um programa ou sistema computacional é a estrutura ou estruturas do sistema, que abrange os componentes de software, as propriedades externamente visíveis desses componentes e as relações entre eles.

A arquitetura não é o software operacional, mas sim, uma representação que nos permite (1) analisar a efetividade do projeto no atendimento dos requisitos declarados, (2) considerar alternativas de arquitetura em um estágio quando realizar mudanças de projeto ainda é relativamente fácil e (3) reduzir os riscos associados à construção do software.

Essa definição enfatiza o papel dos “componentes de software” em qualquer representação de arquitetura. No contexto de projeto da arquitetura, um componente de software pode ser algo tão simples quanto um módulo de programa ou uma classe orientada a objetos, porém ele também pode ser estendido para abranger bancos de dados e “middleware” que possibilita a configuração de uma rede de clientes e servidores. As propriedades dos componentes são aquelas características necessárias para o entendimento de como eles interagem com outros componentes. No nível da arquitetura, propriedades internas (por exemplo, detalhes de um algoritmo) não são especificadas. As relações entre componentes podem ser tão simples quanto a chamada procedural de um módulo a outro ou tão complexo quanto um protocolo de acesso a banco de dados.

Alguns membros da comunidade da engenharia de software (por exemplo, [Kaz03]) fazem uma distinção entre as ações associadas à obtenção de uma arquitetura de software (aquilo que denomino “projeto da arquitetura”) e as ações aplicadas para obter o projeto de software. Conforme observado por um dos revisores desta edição:

“A arquitetura de um sistema constitui uma definição abrangente que descreve sua forma e estrutura — seus componentes e como eles se integram.”

Jerrold Grochow

PONTO-CHAVE

A arquitetura de software deve modelar a estrutura de um sistema, bem como a maneira por meio da qual dados e componentes procedurais colaboram entre si.

“Case-se depressa com sua arquitetura, arrependa-se quando quiser.”

Barry Boehm

Há uma distinta diferença entre os termos arquitetura e projeto. *Projeto* é uma instância de uma *arquitetura* da mesma forma que um objeto é instância de uma classe. Consideremos, por exemplo, a arquitetura cliente/servidor. Podemos projetar um sistema de software centralizado em redes de várias formas diferentes por meio dessa arquitetura usando a plataforma Java (Java EE) ou a plataforma Microsoft (.NET framework). Existe uma arquitetura, porém podem ser criados vários projetos baseados nessa arquitetura. Consequentemente, não podemos confundir “arquitetura” com “projeto”.

WebRef

Links úteis para diversos sites sobre arquitetura de software podem ser encontrados em www2.umassd.edu/SECenter/SAResources.html.

Embora concorde que um projeto de software seja uma instância de uma arquitetura de software específica, os elementos e estruturas definidos como parte de uma arquitetura são a raiz de qualquer projeto que evolui a partir deles. Um projeto se inicia com uma consideração de arquitetura.

Neste livro o projeto de arquitetura de software considera dois níveis da pirâmide de projeto (Figura 8.1) — projeto de dados e projeto da arquitetura. No contexto da discussão anterior, o projeto de dados permite que representemos o componente de dados da arquitetura em sistemas convencionais e as definições de classes (englobando atributos e operações), em sistemas orientados a objetos. O projeto da arquitetura concentra-se na representação da estrutura dos componentes de software, suas propriedades e interações.

9.1.2 Por que a arquitetura é importante?

Em um livro dedicado à arquitetura de software, Bass e seus colegas [Bas03] identificaram três razões-chave da importância da arquitetura de software:

- As representações da arquitetura de software são um facilitador para a comunicação entre todas as partes interessadas no desenvolvimento de um sistema computacional.
- A arquitetura evidencia decisões de projeto iniciais que terão profundo impacto em todo o trabalho de engenharia de software que vem a seguir e, tão importante quanto, no sucesso final do sistema como uma entidade operacional.
- A arquitetura “constitui um modelo relativamente pequeno e intelectualmente compreensível de como o sistema é estruturado e como seus componentes trabalham em conjunto” [Bas03].

O modelo de projeto da arquitetura e os padrões de arquitetura nele contidos são transferíveis. Gêneros, estilos e padrões de arquitetura (Seções 9.2 a 9.4) podem ser aplicados ao projeto de outros sistemas e representam um conjunto de abstrações que permitem aos engenheiros de software descrever arquitetura de modo previsível.

9.1.3 Descrições de arquitetura

Cada um de nós possui uma imagem mental daquilo que a palavra *arquitetura* significa. Entretanto, na realidade, seu significado é diverso para diferentes pessoas. A implicação é que os diferentes interessados verão uma arquitetura sob diversos pontos de vista orientados por conjuntos de interesses distintos. Isso implica que uma descrição de arquitetura é, na verdade, um conjunto de artefatos que refletem diferentes visões do sistema.

Por exemplo, o arquiteto de um importante conjunto comercial tem de trabalhar com uma série de interessados diferentes. O principal interesse do proprietário do conjunto comercial (um dos interessados) é garantir que ele seja esteticamente agradável e que forneça espaço e infraestrutura suficientes para garantir sua lucratividade. Consequentemente, o arquiteto tem de criar uma descrição usando visões de edifício que atendam os interesses do proprietário. Os pontos de vista usados são desenhos tridimensionais do edifício (para ilustrar a visão estética) e um conjunto de plantas bidimensionais para atender à necessidade do interessado em termos de espaço e infraestrutura do conjunto comercial.

Porém, o conjunto comercial tem muitos outros interessados, até mesmo o fabricante de aço de construção especial que fornecerá aço para a estrutura do prédio. Esse fabricante precisa de informações de arquitetura detalhadas sobre o aço de construção que irá su-

“A arquitetura é extremamente importante para ser deixada a cargo de uma única pessoa, independentemente de quão brilhante ela possa ser.”

Scott Ambler

PONTO-CHAVE

O modelo de arquitetura fornece uma visão gestáltica do sistema, permitindo ao engenheiro de software examiná-lo como um todo.



Seu esforço deve focalizar as representações de arquitetura que irão orientar todos os demais aspectos do projeto. Invista o tempo necessário para rever cuidadosamente a arquitetura. Um erro nessa fase terá um impacto negativo no longo prazo.

portar o edifício, incluindo tipos de vigas duplo T, suas dimensões, conectividade, materiais e muitos outros detalhes. Tais interesses são atendidos por diferentes artefatos que representam diferentes visões da arquitetura. Desenhos especializados (um outro ponto de vista) da estrutura de aço de construção do edifício focalizam apenas uma das várias preocupações do fabricante.

A descrição de arquitetura de um sistema baseado em software deve apresentar características análogas às aquelas citadas para o conjunto comercial. Tyree e Akerman [Tyr05] citam isso ao escreverem: “Os desenvolvedores desejam orientação clara e determinada sobre como prosseguir com um projeto. Os clientes querem um entendimento claro sobre as mudanças que devem ocorrer no ambiente e garantias de que a arquitetura atenderá suas necessidades de negócio. Outros arquitetos buscam um entendimento claro dos aspectos mais importantes da arquitetura”. Cada um desses “desejos” se reflete em uma visão diferente representada sob um ponto de vista diferente.

A IEEE Computer Society propôs o IEEE-Std-1471-2000, *Recommended Practice for Architectural Description of Software-Intensive Systems*, [IEE00], com os seguintes objetivos: (1) estabelecer uma definição conceitual e um vocabulário para uso durante o projeto da arquitetura de software, (2) fornecer diretrizes detalhadas para representar uma descrição de arquitetura e (3) encorajar práticas de projeto de arquitetura consistentes.

O padrão IEEE define uma *descrição da arquitetura* (*architectural description, AD*) como “um conjunto de produtos para documentar uma arquitetura”. Uma descrição por si só é representada usando-se várias visões; cada *visão* é “a representação de um sistema como um todo segundo a perspectiva de um conjunto de necessidades [dos interessados] relacionado”. A *visão* é criada de acordo com regras e convenções definidas em um ponto de vista — “uma especificação das convenções para a construção e uso de uma visão” [IEE00]. Uma série de artefatos diferentes utilizados para desenvolver diferentes visões da arquitetura de software é discutida posteriormente, ainda neste capítulo.

9.1.4 Decisões de arquitetura

Cada visão desenvolvida como parte da descrição arquitetural trata uma necessidade específica do interessado. Para desenvolver cada visão (e a descrição arquitetural como um todo), o arquiteto de sistemas considera uma variedade de alternativas e, por fim, decide sobre as características de uma arquitetura específica que melhor atendam à necessidade. Consequentemente, as próprias decisões de arquitetura podem ser consideradas uma visão de arquitetura. As razões pelas quais as decisões foram tomadas fornecem uma visão sobre a estrutura de um sistema e sua adequação às necessidades dos interessados.

Como arquiteto de sistemas, podemos usar o *template* sugerido no quadro para documentar cada decisão importante. Desse modo, fornecemos os fundamentos para nosso trabalho e estabelecemos um registro histórico que pode ser útil quando mudanças de projeto tiverem de ser feitas.

9.2 GÊNEROS DE ARQUITETURA

PONTO-CHAVE

Há uma série de estilos de arquitetura diferentes que poderiam ser aplicados a um gênero específico (também denominado domínio de aplicação).

Embora os princípios fundamentais do projeto da arquitetura se apliquem a todos os tipos de arquitetura, o *gênero* normalmente ditará a abordagem de arquitetura específica para a estrutura que deve ser construída. No contexto de projeto da arquitetura, *gênero* implica uma categoria específica no domínio de software geral. Em cada categoria, pode-se encontrar uma série de subcategorias. Por exemplo, dentro do gênero *edifícios*, poderíamos encontrar os seguintes estilos gerais: casas, condomínios, prédios de apartamentos, conjuntos comerciais, prédios industriais, armazéns e assim por diante. Em cada estilo geral, poderiam ser aplicados estilos mais específicos (Seção 9.3). Cada estilo teria uma estrutura que pode ser descrita usando-se um conjunto de padrões previsíveis.

INFORMAÇÕES

**Template de descrição de decisões de arquitetura**

Cada importante decisão de arquitetura pode ser documentada para posterior revisão pelos interessados que querem entender a descrição da arquitetura proposta. O template apresentado neste quadro é uma versão resumida e adaptada de um gabarito proposto por Tyree e Ackerman [Tyr05].

Problema de projeto:	Descreva os problemas de projeto da arquitetura que devem ser tratados.
Resolução:	Informe a abordagem escolhida para tratar o problema de projeto.
Categoria:	Especifique a categoria de projeto que o problema e a resolução tratam (por exemplo, projeto de dados, estrutura de conteúdo, estrutura de componentes, integração, apresentação).
Hipóteses:	Indique quaisquer hipóteses que ajudem a dar forma à decisão.
Restrições:	Especifique quaisquer restrições do ambiente que auxiliaram a dar forma à decisão (por exemplo, padrões de tecnologia, padrões disponíveis, questões relacionadas ao projeto).

Alternativas:	Descreva brevemente as alternativas de projeto de arquitetura consideradas e por que foram rejeitadas.
Argumento:	Afirme por que escolheu a decisão em relação a outras alternativas.
Implicações:	Indique as consequências de projeto ao tomar a decisão. Como a resolução afeta outras questões do projeto da arquitetura? A resolução irá restringir o projeto de alguma forma?
Decisões relacionadas:	Que outras decisões documentadas estão relacionadas com essa decisão?
Necessidades relacionadas:	Que outros requisitos estão relacionados com essa decisão?
Artefatos:	Indique onde essa decisão irá se refletir na descrição da arquitetura.
Notas:	Faça referência a quaisquer observações feitas pela equipe ou outra documentação utilizada para tomar a decisão.

Em seu manual em constante evolução, *Handbook of Software Architecture* [Boo08], Grady Booch sugere os seguintes gêneros de arquitetura para sistemas baseados em software:

- **Inteligência artificial** — Sistemas que simulam ou ampliam a cognição, locomoção ou outros processos orgânicos humanos.
- **Comercial e sem fins lucrativos** — Sistemas que são fundamentais para a operação de um empreendimento comercial.
- **Comunicações** — Sistemas que fornecem a infraestrutura para a transferência e o gerenciamento de dados, para conectar usuários desses dados ou para apresentar dados na periferia de uma infraestrutura.
- **Autoria de conteúdo** — Sistemas que são utilizados para criar ou manipular artefatos de texto ou multimídia.
- **Dispositivos** — Sistemas que interagem com o mundo físico para fornecer algum serviço para um indivíduo.
- **Esportes e entretenimento** — Sistemas que gerenciam eventos públicos ou que geram uma experiência de entretenimento a um grande grupo.
- **Financeiros** — Sistemas que fornecem a infraestrutura para transferir e administrar dinheiro e outros valores.
- **Jogos** — Sistemas que geram uma experiência de entretenimento para indivíduos ou grupos.
- **Governo** — Sistemas que dão apoio à conduta e operações de uma entidade municipal, estadual, federal, internacional ou outras entidades políticas.
- **Industriais** — Sistemas que simulam ou controlam processos físicos.
- **Legais** — Sistemas que dão apoio ao setor judiciário.
- **Médicos** — Sistemas que diagnosticam ou curam ou contribuem para a pesquisa médica.
- **Militar** — Sistemas para consultas, comunicações, comando, controle e inteligência (C4I), bem como armamento de ataque e defesa.

“Programar sem uma arquitetura ou projeto geral em mente é como explorar uma caverna portando apenas uma lanterna: não sabemos onde estivemos, não sabemos aonde estamos indo e mal sabemos onde estamos no momento.”

Danny Thorpe

- **Sistemas operacionais** — Sistemas que se situam logo acima do hardware para fornecer serviços básicos de software.
- **Plataformas** — Sistemas que se posicionam logo acima dos sistemas operacionais para fornecer serviços avançados.
- **Científicos** — Sistemas que são utilizados para pesquisa e aplicações científicas.
- **Ferramentas** — Sistemas que são utilizados para desenvolver outros sistemas.
- **Transportes** — Sistemas que controlam veículos de navegação, terrestres, aéreos ou espaciais.
- **Serviços públicos** — Sistemas que interagem com outros softwares para fornecer algum serviço.

Sob o ponto de vista do projeto da arquitetura, cada gênero representa um desafio único. Consideremos, por exemplo, a arquitetura de software para um sistema de jogos eletrônicos. Os sistemas de jogos, algumas vezes chamados *aplicações interativas de imersão*, requerem o cálculo de algoritmos intensivos, computação gráfica sofisticada, fontes de dados multimídia *streaming*, interatividade em tempo real via entradas convencionais e não convencionais e uma série de outras necessidades especializadas.

Alexandre Francois [Fra03] sugere uma arquitetura de software para *Immersipresence*¹ que pode ser aplicada a um ambiente de jogos eletrônicos. Ele descreve a arquitetura da seguinte maneira:

SAI (*Software Architecture for Immersipresence*) é um novo modelo de arquitetura de software para projetar, analisar e implementar aplicações realizando processamento paralelo assíncrono distribuído de fluxos de dados genéricos. O objetivo do SAI é fornecer uma estrutura universal para a implementação distribuída de algoritmos e sua fácil integração em sistemas complexos... O modelo de dados subjacente e extensível, bem como o modelo de processamento paralelo assíncrono distribuído (repositório compartilhado e passagem de mensagens) híbrido, possibilita a manipulação natural e eficiente de fluxos de dados genéricos, usando-se bibliotecas ou código nativo indiferentemente. A modularidade do estilo facilita o desenvolvimento, testes e reutilização de código distribuído, bem como o rápido desenvolvimento, integração, manutenção e evolução do sistema.

Uma discussão detalhada do SAI está fora do escopo deste livro. Entretanto, é importante reconhecer que o gênero de sistemas para jogos pode ser tratado com um estilo de arquitetura (Seção 9.3) especificamente desenhado para lidar com as questões de sistemas para jogos eletrônicos. Caso tenha maior interesse, veja [Fra03].

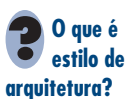
9.3 ESTILOS DE ARQUITETURA

“Por trás da mente de qualquer artista existe um padrão ou tipo de arquitetura.”

G. K. Chesterton

Quando um construtor usa a frase “colonial americano com hall central” para descrever uma casa, a maioria das pessoas familiarizadas com casas nos Estados Unidos será capaz de evocar uma imagem geral de como se parecerá a casa e como provavelmente será a sua planta. O construtor usou um *estilo arquitetônico* como um mecanismo descritivo para diferenciar a casa de outros estilos (por exemplo, casa pré-fabricada sobre uma estrutura de madeira em forma de A, rancho montado sobre uma base elevada, *Cape Cod*). Porém, mais importante ainda, o estilo arquitetônico também é um template para construção. Devem ser definidos mais detalhes da casa, suas dimensões finais especificadas, características personalizadas poderiam ser acrescentadas, também devem ser determinados os materiais de construção, o estilo — uma casa “colonial americano com hall central” — orienta o construtor em seu trabalho.

O software que é criado para sistemas computacionais também apresenta um de muitos estilos de arquitetura. Cada estilo descreve uma categoria de sistema que engloba (1) um conjunto de componentes (por exemplo, um banco de dados, módulos computacionais) que realiza uma função exigida por um sistema; (2) um conjunto de conectores que habilitam a “comunicação, coordenação e cooperação” entre os componentes; (3) restrições que definem como os



1 Francois utiliza o termo *immersipresence* para aplicações interativas e de imersão.

WebRef

Estilos de arquitetura baseados em atributos (sigla ABAS) podem ser utilizados como blocos básicos para as arquiteturas de software. Informações sobre isso podem ser obtidas em www.sei.cmu.edu/architecture/abas.html.

componentes podem ser integrados para formar o sistema; e (4) modelos semânticos que permitem a um projetista compreender as propriedades gerais de um sistema por meio da análise das propriedades conhecidas de suas partes constituintes [Bas03].

Um estilo arquitetural é uma transformação imposta no projeto do sistema inteiro. O objetivo é estabelecer uma estrutura para todos os componentes do sistema. No caso em que uma arquitetura existente deve sofrer um processo de reengenharia (Capítulo 29), a imposição de um estilo de arquitetura resultará em mudanças fundamentais na estrutura do software, incluindo uma nova atribuição da funcionalidade dos componentes [Bos00].

Um padrão de arquitetura, assim como um estilo arquitetural, impõe transformação no projeto de arquitetura. Entretanto, padrão difere de estilo em alguns modos fundamentais: (1) o escopo de um padrão é menos abrangente, concentrando-se em um aspecto da arquitetura e não na arquitetura em sua totalidade; (2) um padrão impõe uma regra sobre a arquitetura, descrevendo como o software irá tratar algum aspecto de sua funcionalidade em termos de infraestrutura (por exemplo, concorrência) [Bos00]; (3) os padrões de arquitetura (Seção 9.4) tendem a tratar questões comportamentais específicas no contexto da arquitetura (por exemplo, como as aplicações em tempo real tratam a sincronização ou as interrupções). Os padrões podem ser usados com um estilo de arquitetura para dar forma à estrutura global de um sistema. Na Seção 9.3.1, consideraremos os padrões e estilos de arquitetura e padrões mais comumente utilizados em software.



Estruturas de arquitetura canônicas

Em essência, a arquitetura de software representa uma estrutura em que algum conjunto de entidades (normalmente denominadas *componentes*) é interligado por um conjunto de relacionamentos definidos (normalmente denominados *conectores*). Tanto os componentes quanto os conectores estão associados a um conjunto de *propriedades* que permitem ao projetista diferenciar os tipos de componentes e conectores que podem ser usados. Mas que tipos de estruturas (componentes, conectores e propriedades) podem ser usados para descrever uma arquitetura? Bass e Kazman [Bas03] sugerem cinco estruturas de arquitetura fundamentais ou canônicas:

Estrutura funcional. Os componentes representam entidades funcionais ou de processamento. Os conectores representam interfaces que fornecem a capacidade de “usar” ou “passar dados para” um componente. As propriedades descrevem a natureza dos componentes e a organização das interfaces.

Estrutura de implementação. “Os componentes podem ser pacotes, classes, objetos, procedimentos, funções, métodos etc., todos os quais são veículos para empacotar funcionalidade em vários níveis de abstração” [Bas03]. Os conectores incluem a habilidade de passar dados e controle, compartilhar dados, “uso” e “instância-de”. As propriedades concentram-se nas características de qualidade (por exemplo, facilidade de

manutenção, reusabilidade) resultantes quando é implementada uma estrutura.

Estrutura de concorrência. Os componentes representam “unidades de concorrência” organizadas como tarefas paralelas ou *threads*. “As relações [conectores] incluem sincronizações-com, é-de maior prioridade-que, envia-dados-a, não é possível-executar-sem e não é possível-executar-com. Entre as propriedades relevantes a essa estrutura temos prioridade, preempção e tempo de execução” [Bas03].

Estrutura física. Essa estrutura é similar ao modelo de implantação desenvolvido como parte do projeto. Os componentes são o hardware físico onde reside o software. Os conectores são as interfaces entre os componentes de hardware e as propriedades tratam a capacidade, largura de banda, desempenho e outros atributos.

Estrutura de desenvolvimento. Essa estrutura define os componentes, artefatos e outras fontes de informação necessárias à medida que a engenharia de software prossegue. Os conectores representam os relacionamentos entre os artefatos, e as propriedades identificam as características de cada item. Cada uma dessas estruturas apresenta visão diferente da arquitetura de software, expondo informações úteis para a equipe de software à medida que a modelagem e a construção prosseguem.

INFORMAÇÕES

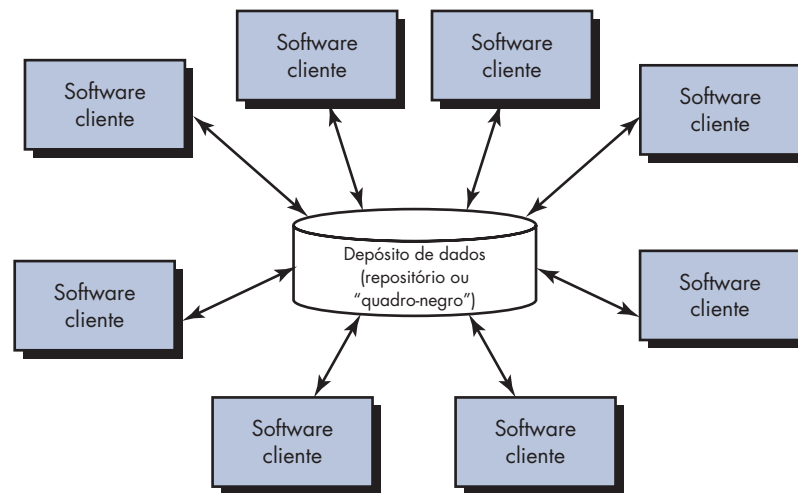
9.3.1 Uma breve taxonomia dos estilos de arquitetura

Embora milhões de sistemas computacionais tenham sido criados ao longo dos últimos 60 anos, a vasta maioria pode ser classificada em um número relativamente pequeno de estilos de arquitetura:

Arquiteturas centralizadas em dados. Um repositório de dados (por exemplo, um arquivo ou banco de dados) reside no centro dessa arquitetura e é em geral acessado por outros componentes que atualizam, acrescentam, eliminam ou de alguma outra maneira modificam dados contidos no repositório. A Figura 9.1 ilustra um estilo centralizado em dados típico. O software-

“O uso de padrões e estilos de projeto é universal nas disciplinas de engenharia.”

Mary Shaw e David Garlan

FIGURA 9.1**Arquitetura centralizada em dados**

cliente acessa um repositório central. Em alguns casos o repositório de dados é passivo. Ou seja, o software-cliente acessa os dados independentemente de quaisquer alterações nos dados ou das ações de outros softwares-clientes. Uma variação dessa abordagem transforma o repositório em um “quadro-negro” que envia notificações ao software-cliente quando os dados de seu interesse mudam.

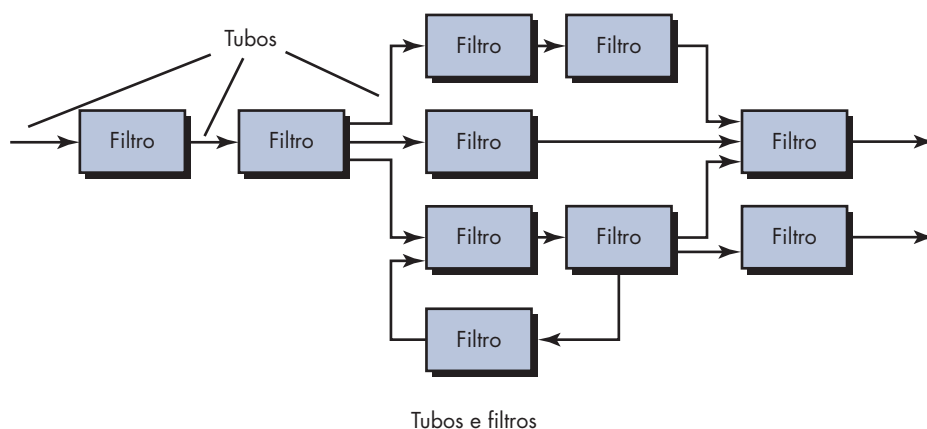
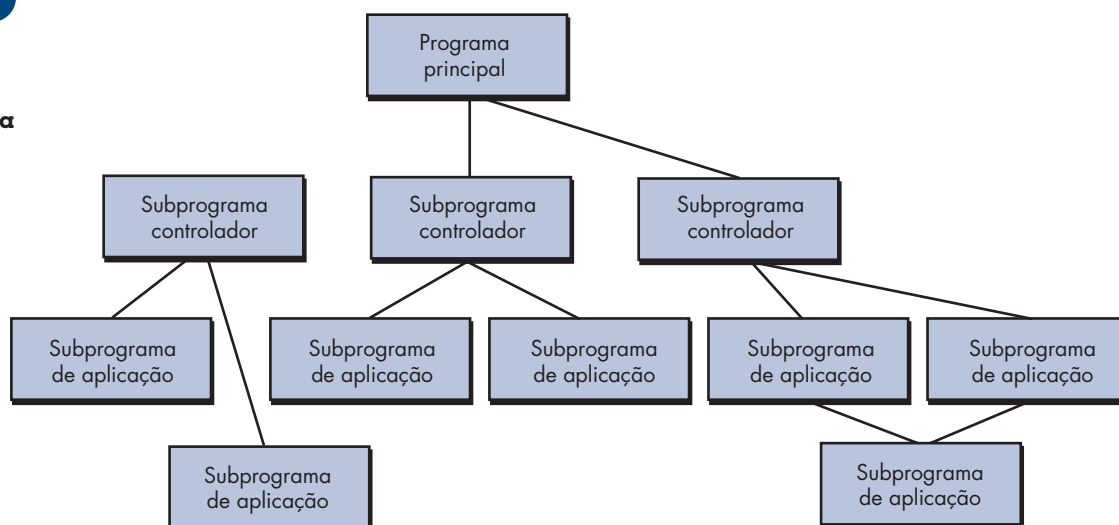
As arquiteturas centralizadas em dados promovem a *integrabilidade* [Bas03]. Isto é, componentes existentes podem ser alterados e novos componentes-clientes acrescentados à arquitetura sem se preocupar com outros clientes (pois os componentes-clientes operam independentemente). Além disso, dados podem ser passados entre os clientes usando o mecanismo de quadro-negro (o componente quadro-negro serve para coordenar a transferência de informações entre os clientes). Os componentes-clientes executam processos de maneira independente.

Arquiteturas de fluxo de dados. Essa arquitetura se aplica quando dados de entrada devem ser transformados por meio de uma série de componentes computacionais ou de manipulação em dados de saída. Um padrão tubos-e-filtro (Figura 9.2) tem um conjunto de componentes, denominado *filtros*, conectados por *tubos* que transmitem dados de um componente para o seguinte. Cada filtro trabalha independentemente dos componentes que se encontram acima e abaixo deles, é projetado para esperar a entrada de dados de determinada forma e produzir saída de dados (para o filtro seguinte) da forma especificada. Entretanto, o filtro não precisa conhecer o funcionamento interno de seus filtros vizinhos.

Se o fluxo de dados ocorrer em uma única linha de transformações, é denominado sequencial por lotes. Essa estrutura aceita um lote de dados e aplica uma série de componentes sequenciais (filtros) para transformá-lo.

Arquiteturas de chamadas e retornos. Esse estilo de arquitetura permite-nos obter uma estrutura de programa relativamente fácil de modificar e aumentar. Existe uma série de subestilos [Bas03] dentro desta categoria:

- *Arquiteturas de programa principal/subprograma.* Essa clássica estrutura de programas decompõe a função em uma hierarquia de controle na qual um programa “principal” invoca uma série de componentes de programa que, por sua vez, podem invocar outros. A Figura 9.3 ilustra uma arquitetura desse tipo.
- *Arquiteturas de chamadas a procedimentos remotos.* Os componentes de uma arquitetura de programas principal /subprogramas são distribuídos ao longo de vários computadores em uma rede.

FIGURA 9.2**Arquitetura de fluxo de dados****FIGURA 9.3****Arquitetura programa principal/subprograma**

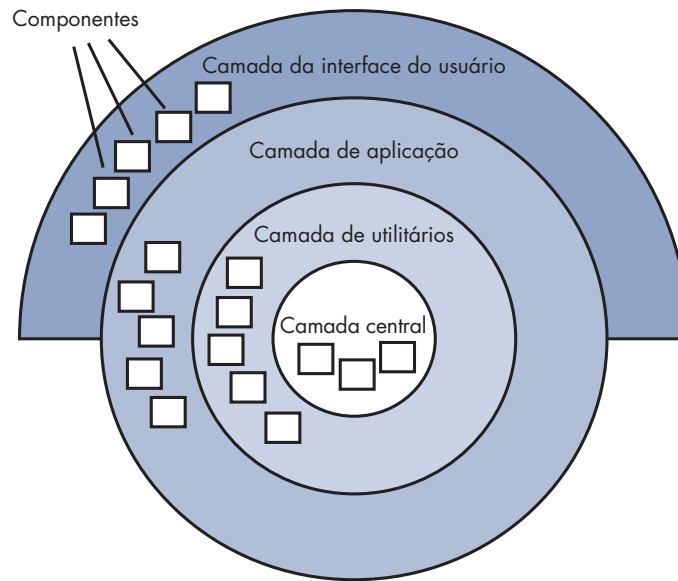
Arquiteturas orientadas a objetos. Os componentes de um sistema encapsulam dados e as operações que devem ser aplicadas para manipular os dados. A comunicação e a coordenação entre componentes são realizadas através da passagem de mensagens.

Arquiteturas em camadas A estrutura básica de uma arquitetura em camadas é ilustrada na Figura 9.4. São definidas várias camadas diferentes, cada uma realizando operações que progressivamente se tornam mais próximas do conjunto de instruções de máquina. Na camada mais externa, os componentes atendem operações de interface do usuário. Na camada mais interna, os componentes realizam a interface com o sistema operacional. As camadas intermediárias fornecem serviços utilitários e funções de software de aplicação.

Esses estilos de arquitetura são apenas um pequeno subconjunto dos disponíveis.² Assim que a engenharia de requisitos revelar as características e restrições do sistema a ser construído, o estilo e/ou combinação de padrões de arquitetura que melhor se encaixa nessas características e restrições pode ser escolhido. Em muitos casos, mais de um padrão poderia ser apropriado, e estilos de arquitetura alternativos podem ser projetados e avaliados. Por exemplo, um estilo em camadas (apropriado para a maioria dos sistemas) pode ser combinado com uma arquitetura centralizada em dados em diversas aplicações de bancos de dados.

2 Veja [Bus07], [Gor06], [Roz05], [Bas03], [Bos00] ou [Hof00] para uma discussão detalhada sobre padrões e estilos de arquitetura.

FIGURA 9.4
Arquitetura em camadas



“Talvez ele esteja no porão. Vou subir e verificar.”

M. C. Escher

9.3.2 Padrões de arquitetura

À medida que o modelo de requisitos é desenvolvido, poderemos perceber que o software deve tratar uma série de problemas mais amplos que envolvem toda a aplicação. Por exemplo, o modelo de requisitos para praticamente qualquer aplicação de comércio eletrônico depara com o seguinte problema: *Como oferecer uma ampla gama de produtos para uma ampla gama de clientes e permitir que esses clientes comprem nossos artigos on-line?*

CASA SEGURA



Escolhendo um estilo de arquitetura

Cena: Sala do Jamie, à medida que a modelagem de projeto começa.

Atores: Jamie e Ed — membros da equipe de engenharia de software do CasaSegura.

Conversa:

Ed (franzindo a testa): Modelamos a função de segurança usando UML... Você sabe, classes, relações, esse tipo de coisas. Portanto, imagino que a arquitetura³ orientada a objetos seja o caminho a seguirmos.

Jamie: Mas...?

Ed: Mas... Tenho dificuldade em visualizar o que é uma arquitetura orientada a objetos. Entendo a arquitetura de chamadas e retornos, uma espécie de hierarquia de processos convencional, mas orientada a objetos... Eu não sei, ela parece um tanto amorfa.

Jamie (sorrindo): Amorfa, uh?

Ed: Isso mesmo... O que eu quis dizer é que não consigo visualizar uma estrutura real, apenas classes de projeto flutuando no ar.

Jamie: Bem, isso não é verdade. Existem hierarquias de classes... Imagine a hierarquia (agregação) que fizemos para o objeto **Planta** [Figura 8.3]. Uma arquitetura orientada a objetos é uma combinação daquela estrutura e as interconexões — sabe, colaborações — entre as classes. Podemos mostrá-la descrevendo completamente os atributos e operações, a troca de mensagens que ocorre e a estrutura das classes.

Ed: Vou gastar uma hora mapeando uma arquitetura de chamadas e retornos; então voltarei e considerarei uma arquitetura orientada a objetos.

Jamie: Doug não terá nenhum problema com isso. Ele me disse que deveríamos considerar alternativas de arquitetura. Por sinal, não há absolutamente nenhuma razão para que essas duas arquiteturas não possam ser usadas de forma combinada.

Ed: Bom. Eu concordo.

³ Pode ser argumentado que a arquitetura do CasaSegura deveria ser considerada em um nível mais elevado do que a arquitetura citada. O CasaSegura possui uma série de subsistemas — funcionalidade de monitoramento da casa, o site de monitoramento da empresa e o subsistema que roda no PC do proprietário do imóvel. Nos subsistemas, processos concorrentes (por exemplo, aqueles para monitorar sensores) e tratamento de eventos são frequentes. Algumas decisões em relação à arquitetura nesse nível são tomadas durante a engenharia de produto, porém o projeto da arquitetura na engenharia de software deve considerar muito bem essas questões.

O modelo de requisitos também define um contexto no qual essa questão deve ser respondida. Por exemplo, uma aplicação de comércio eletrônico que vende equipamento de golfe para clientes irá operar em um contexto diferente daquele de uma aplicação de comércio eletrônico que vende equipamentos industriais de preço elevado para empresas de médio e grande porte. Além disso, um conjunto de limitações e restrições poderia afetar a forma que tratamos o problema a ser resolvido.

Os padrões de arquitetura tratam um problema específico de aplicação em um contexto específico e sob um conjunto de limitações e restrições. O padrão propõe uma solução de arquitetura capaz de servir como base para o projeto da arquitetura.

Citamos anteriormente, neste capítulo, que a maioria das aplicações enquadra-se em um domínio ou gênero específico e que um ou mais estilos de arquitetura poderiam ser apropriados para aquele gênero. Por exemplo, o estilo de arquitetura geral para uma aplicação poderia ser de chamadas e retornos ou orientado a objetos. Porém, nesse estilo, encontraremos um conjunto de problemas comuns que poderiam ser mais bem tratados com padrões de arquitetura específicos. Alguns desses problemas e uma discussão mais completa de padrões de arquitetura são apresentados no Capítulo 12.

9.3.3 Organização e refinamento

Pelo fato de o processo de projeto muitas vezes nos dar uma série de alternativas de arquitetura, é importante estabelecer um conjunto de critérios de projeto que podem ser usados para avaliar o projeto da arquitetura obtido. As seguintes questões [Bas03] dão uma visão mais clara sobre um estilo de arquitetura:

 **Como avaliar um estilo de arquitetura obtido?**

Controle. Como o controle é gerenciado na arquitetura? Existe uma hierarquia de controle distinta e, em caso positivo, qual o papel dos componentes nessa hierarquia de controle? Como os componentes transferem controle no sistema? Como o controle é compartilhado entre os componentes? Qual a topologia de controle (ou seja, a forma geométrica que o controle assume)? O controle é sincronizado ou os componentes operam de maneira assíncrona?

Dados. Como os dados são transmitidos entre os componentes? O fluxo de dados é contínuo ou os objetos de dados são passados esporadicamente para o sistema? Qual o modo de transferência de dados (ou seja, os dados são passados de um componente para outro ou os dados estão disponíveis globalmente para ser compartilhados entre os componentes de sistema)? Os componentes de dados (por exemplo, um quadro-negro ou repositório) existem e, em caso positivo, qual o seu papel? Como os componentes funcionais interagem com os componentes de dados? Os componentes de dados são passivos ou ativos (isto é, o componente de dados interage ativamente com outros componentes do sistema)? Como os dados e controle interagem no sistema?

Essas questões dão ao projetista uma avaliação prévia da qualidade do projeto e formam a base para análise mais detalhada da arquitetura.

9.4 PROJETO DA ARQUITETURA

“Um médico pode enterrar seus erros, mas um arquiteto pode apenas aconselhar seu cliente a plantar videiras.”

Frank Lloyd Wright

Quando o projeto da arquitetura se inicia, o software a ser desenvolvido deve ser colocado no contexto — o projeto deveria definir as entidades externas (outros sistemas, dispositivos, pessoas) com as quais o software interage e a natureza da interação. Essas informações em geral podem ser obtidas do modelo de requisitos e todas as demais coletadas durante a engenharia de requisitos. Uma vez que o contexto é modelado e todas as interfaces de software externas tenham sido descritas, podemos identificar um conjunto de arquétipos arquiteturais. *Arquétipo* é uma abstração (similar a uma classe) que representa um elemento do comportamento do sistema. O conjunto de arquétipos fornece uma coleção de abstrações que deve ser modelada arquiteturalmente caso o sistema tenha de ser construído, porém os arquétipos em si não fornecem detalhes de implementação suficientes. Consequentemente, o projetista especifica uma

estrutura do sistema por meio da definição e refinamento de componentes de software que implementam cada arquétipo. Esse processo continua iterativamente até que uma estrutura de arquitetura completa tenha sido obtida. Nas seções seguintes examinaremos cada uma dessas tarefas de projeto da arquitetura com um pouco mais de detalhes.

9.4.1 Representação do sistema no contexto

PONTO-CHAVE

O contexto de arquitetura representa como o software interage com entidades externas a seus limites.

Como os sistemas operam entre si?

No nível de projeto da arquitetura, um arquiteto de software usa um *diagrama de contexto arquitetural* (*architectural context diagram, ACD*) para modelar a maneira pela qual o software interage com entidades externas a seus limites. A estrutura genérica do diagrama de contexto arquitetural é ilustrada na Figura 9.5.

Referindo-se à figura, os sistemas que interoperam com o *sistema-alvo* (o sistema para o qual um projeto da arquitetura deve ser desenvolvido) são representados como

- *Sistemas superiores* — aqueles sistemas que usam o sistema-alvo como parte de algum esquema de processamento de mais alto nível.
- *Sistemas subordinados* — aqueles sistemas que são utilizados pelo sistema-alvo e fornecem dados ou processamento necessários para completar a funcionalidade do sistema-alvo.
- *Sistemas de mesmo nível (pares)* — aqueles sistemas que interagem em uma base par-a-par (ou seja, as informações são produzidas ou consumidas pelos pares e sistema-alvo).
- *Atores* — entidades (pessoas, dispositivos) que interagem com o sistema-alvo através da produção ou consumo de informações necessárias para o processamento.

Cada uma dessas entidades externas se comunica com o sistema-alvo por meio de uma interface (os pequenos retângulos sombreados). Para ilustrarmos o uso do ACD, consideremos a função de segurança residencial do produto *CasaSegura*. O controlador geral do produto *CasaSegura* e o sistema baseado na Internet são ambos superiores em relação à função de segurança e são mostrados acima da função na Figura 9.6. A função de vigilância é um *sistema de mesmo nível* e utiliza (é utilizado por) a função de segurança residencial em versões posteriores do produto. O proprietário do imóvel e os painéis de controle são os atores que agem tanto como produtores quanto consumidores de informações usadas/produzidas pelo software de segurança. Por fim, são utilizados sensores pelo software de segurança e mostrados como subordinados a ele.

FIGURA 9.5

Diagrama de contexto arquitetural

Fonte: Adaptado de (Bos00)

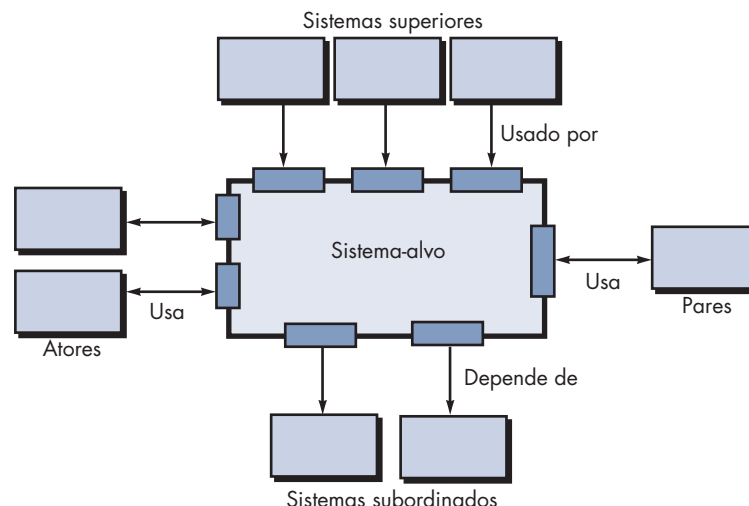
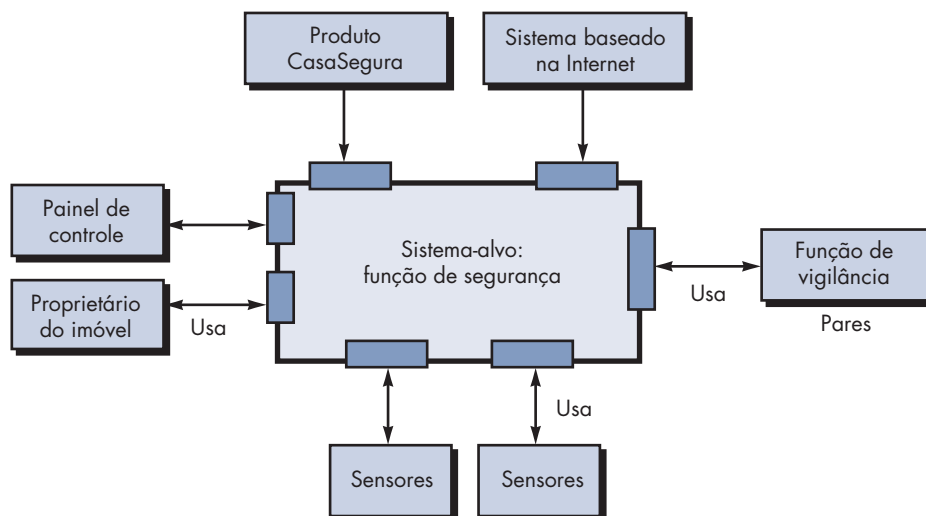


FIGURA 9.6

Diagrama de contexto arquitetural para a função de segurança do CasaSegura



Como parte do projeto da arquitetura, os detalhes de cada interface da Figura 9.6 teriam de ser especificados. Todos os dados que fluem para dentro e para fora do sistema-alvo têm de ser identificados neste estágio.

9.4.2 Definição de arquétipos

Arquétipo é uma classe ou padrão que representa uma abstração central crítica para o projeto de uma arquitetura para o sistema-alvo. Em geral, é necessário um conjunto relativamente pequeno de arquétipos para projetar até mesmo sistemas relativamente complexos. A arquitetura do sistema-alvo é composta desses arquétipos, que representam elementos estáveis da arquitetura, porém podem ser instanciados de várias maneiras tomando como base o comportamento do sistema.

Em muitos casos, os arquétipos podem ser derivados examinando-se as classes de análise definidas como parte do modelo de requisitos. Prosseguindo com a discussão da função de segurança domiciliar do *CasaSegura*, poderíamos definir os seguintes arquétipos:

- **Nó.** Representa um conjunto coeso de elementos de entrada e saída da função de segurança domiciliar. Por exemplo, um nó poderia ser composto por (1) vários sensores e (2) uma série de indicadores (de saída) de alarme.
- **Detector.** Abstração que engloba todos os equipamentos de sensoriamento que alimentam o sistema-alvo com informações.
- **Indicador.** Abstração que representa todos os mecanismos (por exemplo, sirene de alarme, luzes intermitentes, campainha) para indicar a ocorrência de condição de alarme.
- **Controlador.** Abstração que representa o mecanismo que permite armar ou desarmar um nó. Se os controladores residirem em uma rede, eles têm a habilidade de se comunicarem entre si.

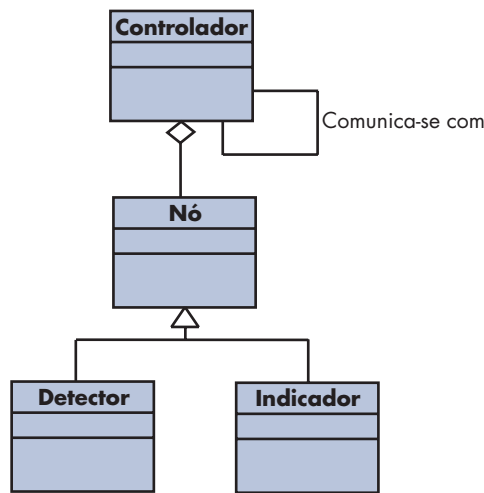
Cada um dos arquétipos é representado usando-se a notação UML conforme indicado na Figura 9.7. Recorde-se que os arquétipos formam a base para a arquitetura, mas são as abstrações que devem ser refinadas à medida que o projeto da arquitetura prossegue. Por exemplo, **Detector** poderia ser refinado em uma hierarquia de classes de sensores.

PONTO-CHAVE

Arquétipos são os blocos básicos abstratos de um projeto da arquitetura.

FIGURA 9.7

Relacionamentos UML para os arquétipos da função de segurança do CasaSegura
 Fonte: Adaptado de (Bos00)



9.4.3 Refinamento da arquitetura em componentes

À medida que a arquitetura de software é refinada em componentes, a estrutura do sistema começa a emergir. Mas como os componentes são escolhidos? Para podermos responder a essa pergunta, começamos pelas classes descritas como parte do modelo de requisitos.⁴ As classes de análise representam entidades no domínio de aplicação que deve ser tratado na arquitetura de software. Portanto, o domínio de aplicação é uma fonte para a derivação e o refinamento de componentes. Outra fonte é o domínio da infraestrutura. A arquitetura deve acomodar muitos componentes de infraestrutura que possibilitem componentes de aplicação, mas que não tenham nenhuma ligação de atividade com o domínio de aplicação. Por exemplo, componentes de gerenciamento de memória, componentes de comunicação, componentes de bancos de dados e componentes de gerenciamento de tarefas em geral são integrados à arquitetura do software.

As interfaces representadas no diagrama de contexto arquitetural (Seção 9.4.1) implica um ou mais componentes especializados que processam os dados que fluem pela interface. Em alguns casos (por exemplo, uma interface gráfica do usuário), tem de ser projetada uma arquitetura de subsistemas completa com vários componentes.

Continuando com o exemplo da função de segurança domiciliar do *CasaSegura*, poderíamos definir o conjunto de componentes de alto nível que atende à seguinte funcionalidade:

- *Gerenciamento da comunicação externa* — coordena a comunicação da função de segurança com entidades externas como sistemas baseados na Internet e notificação externa de alarme.
- *Processamento de painel de controle* — gerencia toda a funcionalidade do painel de controle.
- *Gerenciamento de detectores* — coordena o acesso a todos os detectores conectados ao sistema.
- *Processamento de alarme* — verifica e atua sobre todas as condições de alarme.

Cada um dos componentes de alto nível teria de ser elaborado de forma iterativa e posicionado na arquitetura global do *CasaSegura*. As classes de projeto (com atributos e operações

“A estrutura de um sistema de software fornece o meio no qual o código nasce, amadurece e morre. Um hábitat bem projetado possibilita a evolução bem-sucedida de todos os componentes necessários em um sistema de software.”

R. Pattis

⁴ Se optar-se por uma abordagem convencional (não orientada a objetos), os componentes são obtidos do modelo de fluxo de dados. Discutiremos brevemente esse método na Seção 9.6.

apropriados) seriam definidas para cada um deles. Entretanto, é importante notar que os detalhes de projeto de todos os atributos e operações não seriam especificados até se atingir o projeto de componentes (Capítulo 10).

A estrutura global de arquitetura (representada na forma de um diagrama de componentes UML) é ilustrada na Figura 9.8. As transações são capturadas por *gerenciamento de comunicação externa* à medida que se deslocam de componentes que processam a interface gráfica do usuário do *CasaSegura* e a interface com a Internet. Tais informações são gerenciadas por um componente executivo do *CasaSegura* que seleciona a função do produto apropriada (no caso, segurança). O componente *processamento do painel de controle* interage com o proprietário do imóvel para armar/desarmar a função de segurança. O componente *gerenciamento de detectores* faz uma sondagem nos sensores para detectar condição de alarme, e o componente de *processamento de alarme* gera uma saída quando o alarme é detectado.

9.4.4 Descrição das instâncias

O projeto da arquitetura modelado até este ponto ainda é relativamente de alto nível. O contexto do sistema foi representado, arquétipos que indicam importantes abstrações contidas no domínio do problema foram definidos, a estrutura global do sistema está evidente e os principais componentes de software foram identificados. Entretanto, um maior refinamento (recorde-se que todo projeto é iterativo) ainda é necessário.

Para tanto, é desenvolvida uma instância real da arquitetura. Queremos dizer com isso que a arquitetura é aplicada a um problema específico com o intuito de demonstrar que a estrutura e os componentes são apropriados.

A Figura 9.9 ilustra uma instância da arquitetura do *CasaSegura* para o sistema de segurança. Os componentes da Figura 9.8 são elaborados para indicar mais detalhes. Por exemplo, o componente *gerenciamento de detectores* interage com o *componente de infraestrutura agendador* que implementa a sondagem (*pooling*) de cada objeto *sensor* usado pelo sistema de segurança. Elaboração similar é feita para cada um dos componentes representados na Figura 9.8.

FIGURA 9.8

Estrutura de arquitetura global para o sistema *CasaSegura* com os componentes de alto nível

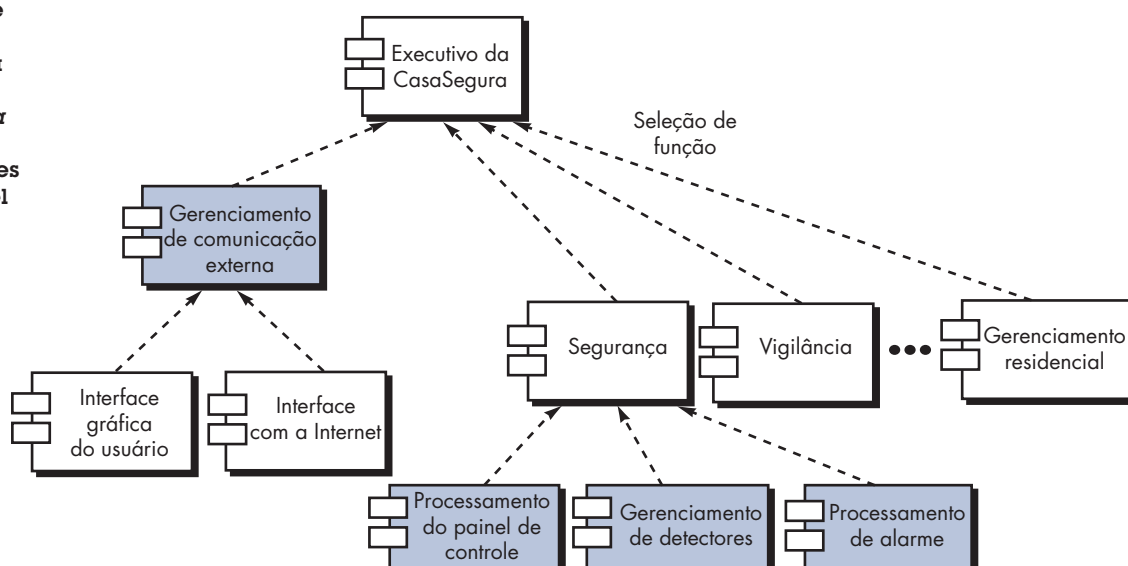
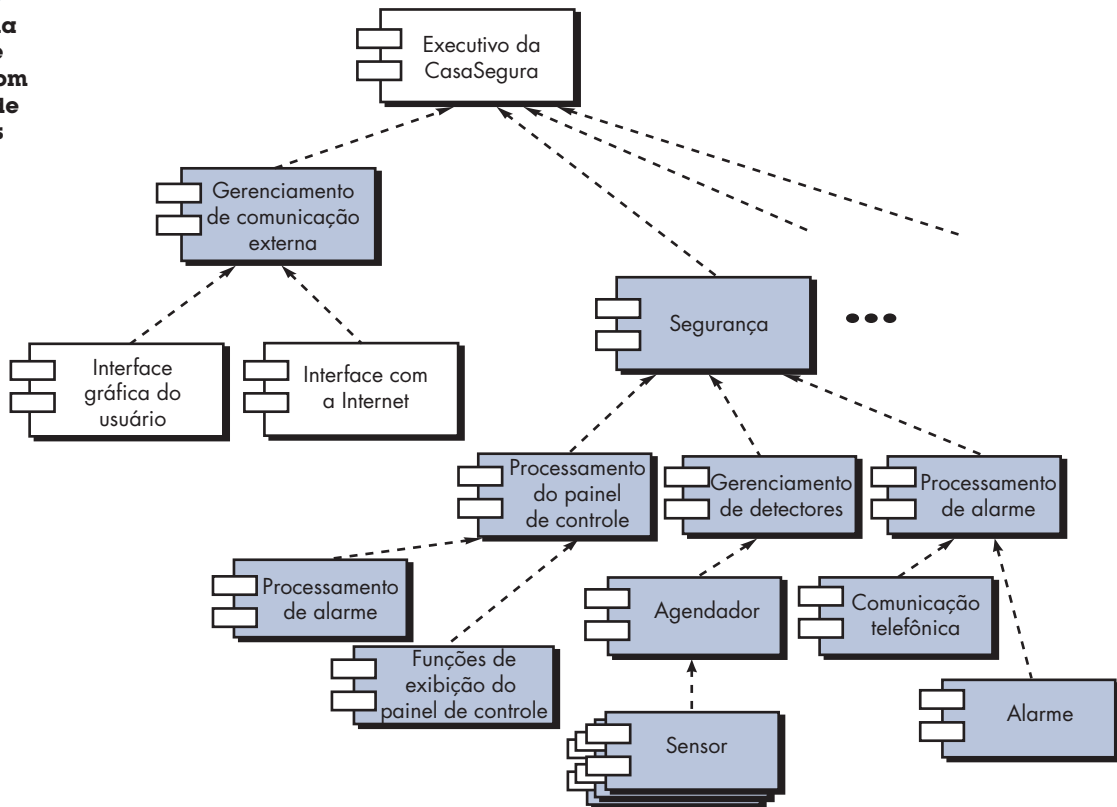


FIGURA 9.9

Uma instância da função de segurança com elaboração de componentes



FERRAMENTAS DO SOFTWARE



Projeto da arquitetura

Objetivo: As ferramentas de projeto da arquitetura modelam a estrutura global do software, representando a interface com componentes, as dependências e relações, bem como as interações.

Mecânica: A mecânica das ferramentas varia. Na maioria dos casos, a capacidade de projeto da arquitetura faz parte da funcionalidade fornecida por ferramentas automatizadas para análise e modelagem de projeto.

Ferramentas representativas:⁵

Adalon, desenvolvida pela Synthis Corp. (www.synthis.com), é uma ferramenta de projeto especializada para

o projeto e a construção de arquiteturas específicas de componentes baseados na Web.

ObjectiF, desenvolvida pela microTOOL GmbH (www.microtool.de/objectiF/en/), é uma ferramenta de projeto baseada em UML que conduz a arquiteturas (por exemplo, Coldfusion, J2EE, Fusebox) suscetíveis à engenharia de software baseada em componentes (Capítulo 29).

Rational Rose, desenvolvida pela Rational (www-306.ibm.com/software/rational/), é uma ferramenta de projeto baseada em UML que oferece suporte a todos os aspectos do projeto da arquitetura.

9.5 AVALIAÇÃO DE PROJETOS DE ARQUITETURAS ALTERNATIVOS

Em seu livro sobre avaliação de arquiteturas de software, Clements e seus colegas [Cle03] afirmam:

Grosso modo, arquitetura é uma aposta, uma aposta no sucesso de um sistema. Não seria ótimo saber antecipadamente se apostamos em um vencedor, em vez de ter de esperar até que o sistema esteja

⁵ As ferramentas aqui apresentadas não significam um aval, mas sim, uma amostra dessa categoria. Na maioria dos casos, seus nomes são marcas registradas pelos respectivos desenvolvedores.

quase completo antes de saber se ele irá atender ou não seus requisitos? Caso esteja adquirindo um sistema ou pagando por seu desenvolvimento, você não gostaria de ter alguma certeza de que foi iniciado de maneira correta? Caso seja o próprio arquiteto de sistemas, você não gostaria de ter um bom meio de validar suas intuições e experiência, para poder dormir à noite sabendo que a confiança colocada em seu projeto é bem fundamentada?

De fato, respostas a essas questões teriam valor. O projeto resulta em uma série de alternativas de arquitetura, cada uma das quais avaliadas para determinar qual delas é a mais apropriada para o problema a ser resolvido. Nas seções a seguir, apresentaremos duas abordagens diferentes para a avaliação de projetos de arquitetura alternativos. A primeira usa um método iterativo para avaliar prós e contras de projeto. A segunda abordagem aplica uma técnica pseudoquantitativa para avaliar a qualidade de um projeto.

9.5.1 Um método de análise dos prós e contras de uma arquitetura

O SEI (Software Engineering Institute) desenvolveu um *método de análise dos prós e contras de uma arquitetura* (ATAM, *architecture trade-off analysis method*, ATAM) [Kaz98] que estabelece um processo de avaliação iterativa de arquiteturas de software. As atividades de análise de projeto a seguir são realizadas iterativamente:

1. *Coletar cenários.* É desenvolvido um conjunto de casos de uso (Capítulos 5 e 6) para representar o sistema sob o ponto de vista do usuário.
2. *Elicitar requisitos, restrições e descrição do ambiente.* Essas informações são determinadas como parte da engenharia de requisitos e usadas para haver certeza de que todas as necessidades dos interessados foram atendidas.
3. *Descrever os estilos/padrões de arquitetura escolhidos para lidar com os cenários e requisitos.* O(s) estilo(s) de arquitetura deveria(m) ser descrito(s) usando uma das seguintes visões de arquitetura:
 - *Visão de módulos* para a análise de atribuições de trabalho com componentes e o grau que foi atingido pelo encapsulamento de informações.
 - *Visão de processos* para a análise do desempenho do sistema.
 - *Visão de fluxo de dados* para análise do grau que a arquitetura atende às necessidades funcionais.
4. *Avaliar atributos de qualidade considerando cada atributo isoladamente.* O número de atributos de qualidade escolhidos para análise é em função do tempo disponível para revisão e o grau em que os atributos de qualidade são relevantes para o sistema em questão. Entre os atributos de qualidade para avaliação de projetos de arquitetura temos confiabilidade, desempenho, segurança, facilidade de manutenção, flexibilidade, facilidade de teste, portabilidade, reusabilidade e interoperabilidade.
5. *Identificar a sensibilidade dos atributos de qualidade em relação a vários atributos de arquitetura para um estilo de arquitetura específico.* Isso pode ser obtido fazendo-se pequenas alterações na arquitetura e na determinação de quão sensível um atributo de qualidade, digamos o desempenho, é em relação a uma mudança. Quaisquer atributos afetados significativamente por uma variação na arquitetura são denominados *pontos de sensibilidade*.
6. *Criticar arquiteturas candidatas (desenvolvidas na etapa 3) usando a análise de sensibilidade conduzida na etapa 5.* O SEI descreve esse método da seguinte maneira [Kaz98]:

Uma vez determinados os pontos de sensibilidade da arquitetura, encontrar os prós e os contras é simplesmente a identificação de elementos de arquitetura para os quais vários atributos são sensíveis. Por exemplo, o desempenho de uma arquitetura cliente/servidor poderia ser altamente sensível ao número de servidores (o desempenho aumenta, em algum intervalo, aumentando-se o número de servidores)... O número de servidores é então um ponto de avaliação com relação a essa arquitetura.

As seis etapas representam a primeira iteração ATAM. Baseado nos resultados das etapas 5 e 6, algumas alternativas de arquitetura poderiam ser eliminadas, uma ou mais arquiteturas

WebRef

Informações detalhadas sobre o ATAM podem ser obtidas em:

www.sei.cmu.edu/activities/architecture/ata_method.html

CASA SEGURA



Avaliação de arquiteturas

Cena: Sala de Doug Miller à medida que a modelagem do projeto da arquitetura prossegue.

Atores: Vinod, Jamie e Ed — membros da equipe de engenharia de software do CasaSegura e Doug Miller, gerente do grupo de engenharia de software.

Conversa:

Doug: Sei que vocês estão obtendo uma série de arquiteturas diferentes para o produto CasaSegura, e isso é bom. Acredito que a minha pergunta seja “como iremos escolher a melhor delas”?

Ed: Estou trabalhando em um estilo de chamadas e retornos e depois eu ou o Jamie vamos derivar uma arquitetura orientada a objetos.

Doug: Certo, e como fazemos essa escolha?

Jamie: Fiz um curso de projeto no meu último ano de faculdade e me recordo que há uma série de maneiras para fazer isso.

Vinod: Realmente, porém elas são um tanto acadêmicas. Veja, acredito que possamos fazer nossa avaliação e escolher a arquitetura correta empregando casos de uso e cenários.

Doug: Isso não é a mesma coisa?

Vinod: Não quando se trata de avaliação de arquiteturas. Já temos um conjunto completo de casos de uso. Portanto, aplica-

mos cada um deles a ambas as arquiteturas e verificamos como o sistema reage, como os componentes e conectores funcionam no contexto dos casos de uso.

Ed: É uma boa ideia. Garante que não nos esqueçamos de nada.

Vinod: É verdade, mas eles também nos informam se o projeto da arquitetura é complicado ou não, se o sistema tem ou não de se desdobrar para conseguir realizar sua tarefa.

Jamie: Cenários não é justamente um outro nome para casos de uso.

Vinod: Não, neste caso um cenário implica algo diferente.

Doug: Você está se referindo a um cenário de qualidade ou um cenário de mudanças, certo?

Vinod: Isso mesmo. O que fazemos é procurar novamente os interessados e perguntar a eles como provavelmente o CasaSegura irá se modificar ao longo dos próximos, digamos, três anos. Sabe, novas versões, recursos, esse tipo de coisa. Construímos um conjunto de cenários de mudanças. Também desenvolvemos um conjunto de cenários de qualidade que definem os atributos que gostaríamos de ver na arquitetura do software.

Jamie: E os aplicamos às alternativas.

Vinod: Exatamente. O estilo que se comportar melhor nos casos de uso e cenários será o escolhido.

remanescentes poderiam ser modificadas e representadas de forma mais detalhada e então as etapas do ATAM seriam reaplicadas.⁶

9.5.2 Complexidade da arquitetura

Uma técnica útil para avaliar a complexidade geral de uma arquitetura proposta é considerar as dependências entre os componentes contidos na arquitetura. Tais dependências são orientadas por fluxo de controle/informações no sistema. Zhao [Zha98] sugere três tipos de dependências:

As dependências de compartilhamento representam relações de dependência entre consumidores que usam o mesmo recurso ou produtores que produzem para os mesmos consumidores. Por exemplo, para dois componentes **u** e **v**, se **u** e **v** fazem referência aos mesmos dados globais, então existe uma relação de dependência compartilhada entre **u** e **v**.

As dependências de fluxo representam relações de dependência entre produtores e consumidores de recursos. Por exemplo, para dois componentes **u** e **v**, se **u** deve completar antes que o controle passe para **v** (pré-requisito), ou se **u** se comunica com **v** através de parâmetros, então existe uma relação de dependência de fluxos entre **u** e **v**.

As dependências de restrição representam restrições no fluxo relativo de controle entre um conjunto de atividades. Por exemplo, para dois componentes **u** e **v**, **u** e **v** não podem ser executados ao mesmo tempo (exclusão mútua), então existe aí uma relação de dependência de restrição entre **u** e **v**.

As dependências de compartilhamento e de fluxo citadas por Zhao são similares ao conceito de acoplamento discutido no Capítulo 8. Acoplamento é um importante conceito de projeto apli-

⁶ O Método de Análise de Arquitetura via Software (Software Architecture Analysis Method, SAAM) é uma alternativa ao ATAM e vale a pena ser examinado por leitores interessados em análise de arquiteturas. Um artigo sobre o SAAM pode ser baixado de www.sei.cmu.edu/publications/articles/saam-metho-propert-sas.html.

cável nos níveis de arquitetura e de componente. Métricas simples para avaliar o acoplamento são discutidas no Capítulo 23.

9.5.3 Linguagens de descrição da arquitetura

O arquiteto de uma casa possui um conjunto de notações e ferramentas padronizadas que permitem que o projeto seja representado de uma forma inequívoca e compreensível. Embora o arquiteto de software possa desenhar em notação UML, outras formas esquemáticas e algumas ferramentas relacionadas, há a necessidade de uma abordagem mais formal para a especificação de um projeto da arquitetura.

A *linguagem de descrição da arquitetura* (*architectural description language, ADL*) fornece uma sintaxe e semântica para descrever uma arquitetura de software. Hofmann e seus colegas [Hof01] sugerem que uma ADL deveria fornecer ao projetista a habilidade de decompor componentes da arquitetura, compor componentes individuais em blocos arquiteturais maiores e representar interfaces (mecanismos de conexão) entre componentes. Uma vez estabelecidas técnicas descritivas e baseadas em linguagens para o projeto da arquitetura, é mais provável que métodos de avaliação de arquitetura mais efetivos serão estabelecidos à medida que o projeto evolui.



Linguagens de descrição da arquitetura

O resumo a seguir de uma série de importantes ADLs foi preparado por Rickard Land [Lan02] e é reimpresso com a permissão do autor. Deve-se salientar que as primeiras cinco ADLs listadas foram desenvolvidas para fins de pesquisa e não são produtos comerciais.

Rapide (<http://poset.stanford.edu/rapide/>) baseia-se no conceito de conjuntos ordenados parciais e, portanto, introduz construtos de programação bastante novos (po- rém aparentemente poderosos).

UniCon (www.cs.cmu.edu/~UniCon) é “uma linguagem de descrição da arquitetura destinada a auxiliar os projetistas a definir arquiteturas de software em termos de abstrações que achem úteis”.

Aesop (www.cs.cmu.edu/~able/aesop/) trata o problema de reutilização de estilos. Com a Aesop, é pos-

sível definir estilos e usá-los ao construir um sistema real.

Wright (www.cs.cmu.edu/~able/wright/) é uma linguagem formal que inclui os seguintes elementos: *componentes* com *portas*, *conectores* com *papéis*, e *cola* para ligar papéis às portas. Os estilos de arquitetura podem ser formalizados na linguagem através de predicados permitindo, portanto, verificações estáticas para determinar a consistência e a completude de uma arquitetura.

Acme (www.cs.cmu.edu/~acme/) pode ser vista como uma ADL de segunda geração, já que seu intuito é identificar um tipo de mínimo denominador comum para ADLs.

UML (www.uml.org/) inclui muitos dos artefatos necessários para descrições de arquiteturas — processos, nós, visões etc. Para descrições informais, a UML é adequada apenas pelo fato de ser um padrão largamente compreendido. Falta-lhe, entretanto, o poder necessário para uma descrição adequada de uma arquitetura.

FERRAMENTAS DO SOFTWARE

9.6 MAPEAMENTO DE ARQUITETURA UTILIZANDO FLUXO DE DADOS

Os estilos de arquitetura discutidos na Seção 9.3.1 representam arquiteturas radicalmente diferentes. Portanto, não é de surpreender se um mapeamento abrangente que realize a transição do modelo de requisitos para uma série de estilos de arquitetura não exista. De fato, não existe nenhum mapeamento prático para alguns estilos de arquitetura, e o projetista deve abordar a tradução de requisitos em projeto para esses estilos usando as técnicas discutidas na Seção 9.4.

Para ilustrarmos uma abordagem para mapeamento de arquitetura, consideremos a arquitetura de chamadas e retornos — uma estrutura especial extremamente comum para muitos tipos de sistemas. A arquitetura de chamadas e retornos pode residir no interior de outras arquiteturas mais sofisticadas discutidas anteriormente neste capítulo. Por exemplo, a arquitetura de um ou mais componentes de uma arquitetura cliente/servidor poderia ser de chamadas e retornos.

Uma técnica de mapeamento, chamada *projeto estruturado* [You79], é muitas vezes caracterizada como um fluxo de dados – método de projeto orientado, pois fornece uma transição

conveniente de um diagrama de fluxo de dados (Capítulo 7) para uma arquitetura de software.⁷ A transição de fluxo de informações (representado na forma de um DFD, diagrama de fluxo de dados) para estrutura de programas é obtida como parte de um processo de seis etapas: (1) é estabelecido o tipo de fluxo de informações, (2) são indicadas as fronteiras do fluxo, (3) o DFD é mapeado em uma estrutura de programas, (4) é definida uma hierarquia de controle, (5) a estrutura resultante é refinada usando-se heurística e medidas de projeto e, por fim, (6) a descrição da arquitetura é refinada e elaborada.

Como um rápido exemplo de mapeamento de fluxo de dados, apresentamos um mapeamento “de transformação” para uma pequena parte da função de segurança do *CasaSegura*.⁸ Para realizar o mapeamento, o tipo de fluxo de informações deve ser determinado. Um tipo de fluxo de informações é denominado *fluxo de transformação* e que apresenta uma característica linear. Os dados fluem para dentro do sistema ao longo de uma *trajetória de fluxo de entrada* na qual são transformados de uma representação do mundo exterior em uma forma internalizada. Assim que tiverem sido internalizados, são processados em um *centro de transformação*. Por fim, fluem para fora do sistema ao longo de uma *trajetória de fluxo de saída* que transforma os dados para a forma do mundo exterior.⁹

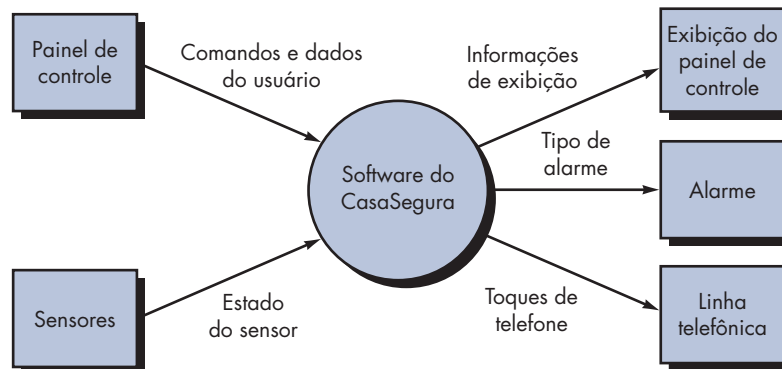
9.6.1 Mapeamento de transformação

Mapeamento de transformação é um conjunto de etapas de projeto que permite que um DFD com características de fluxo de transformação seja mapeado em um estilo de arquitetura específico. Para ilustrarmos essa abordagem, consideraremos mais uma vez a função de segurança do *CasaSegura*.¹⁰ Um elemento do modelo de análise é um conjunto de diagramas de fluxo de dados que descreve o fluxo de informações dentro da função de segurança. Para mapearmos os diagramas de fluxo de dados em uma arquitetura de software, iniciariamos com as seguintes etapas de projeto:

Passo 1. Revisar o modelo fundamental do sistema. O modelo fundamental do sistema ou diagrama de contexto representa a função de segurança como uma única transformação, representando os produtores e consumidores externos de dados que fluem para dentro e para fora da função. A Figura 9.10 representa um modelo de contexto de nível 0, e a Figura 9.11 mostra o fluxo de dados refinado para a função de segurança.

FIGURA 9.10

DFD de nível de contexto para a função de segurança do CasaSegura



⁷ Deve-se notar que outros elementos do modelo de requisitos também são usados durante o método de mapeamento.

⁸ Uma discussão mais detalhada de projeto estruturado é apresentada no site que acompanha este livro.

⁹ Outro importante tipo de fluxo de informações, o *fluxo de transações*, não é considerado nesse exemplo, mas é tratado em um exemplo de projeto estruturado apresentado no site que acompanha este livro.

¹⁰ Consideraremos apenas a parte da função de segurança do *CasaSegura* que utiliza o painel de controle. Outras características discutidas ao longo deste livro não são consideradas aqui.



Se o DFD for refinado ainda mais nesta ocasião, tente ao máximo obter bolhas que exibam alta coesão.

Passo 2. Revisar e refinar diagramas de fluxo de dados para o software. As informações obtidas do modelo de requisitos são refinadas para produzir detalhes significativos. Por exemplo, o DFD de nível 2 para *monitorar sensores* (Figura 9.12) é examinado, e um diagrama de fluxo de dados nível 3 é obtido, conforme mostrado na Figura 9.13. No nível 3, cada transformação no diagrama de fluxo de dados exibe uma coesão relativamente elevada (Capítulo 8). Isto é, o processo correspondente a uma transformação realiza uma única função distinta que pode ser implementada na forma de um componente no software *CasaSegura*. Consequentemente, o DFD da Figura 9.13 contém detalhes suficientes para uma “primeira aproximação” do projeto de arquitetura para o subsistema *monitorar sensores*, e prosseguirmos sem refinamentos adicionais.

FIGURA 9.11

DFD de nível 1 para a função de segurança do CasaSegura

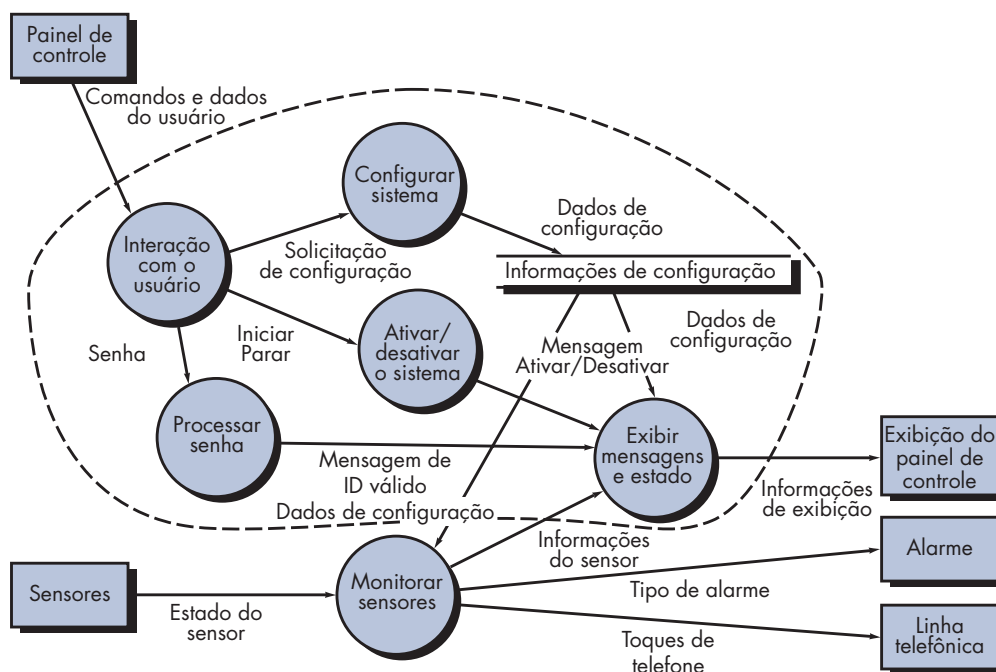
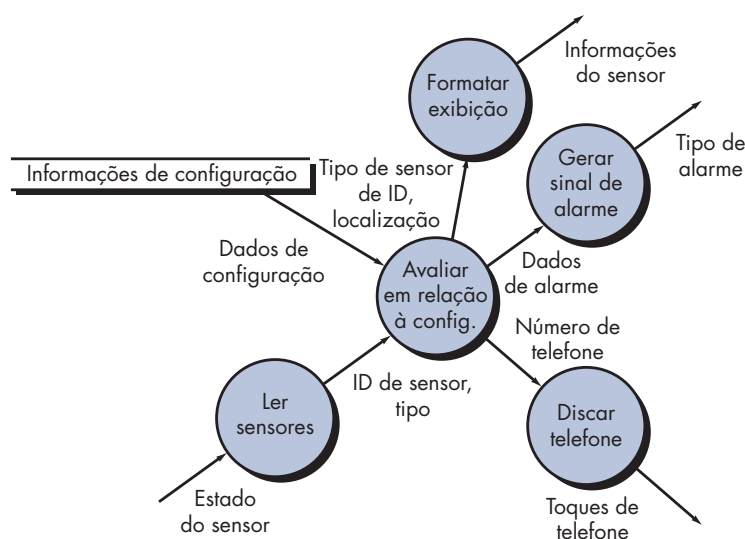


FIGURA 9.12

DFD de nível 2 que refina a transformação de monitorar sensores



PONTO-CHAVE

Muitas vezes encontraremos ambos os tipos de fluxo de dados no interior do mesmo modelo orientado a fluxos. Os fluxos são subdivididos e a estrutura de programas é obtida usando-se o mapeamento apropriado.



Varie a posição das fronteiras dos fluxos em uma tentativa de explorar estruturas alternativas de programas. Isso toma muito pouco tempo e dá uma visão importante do processo.

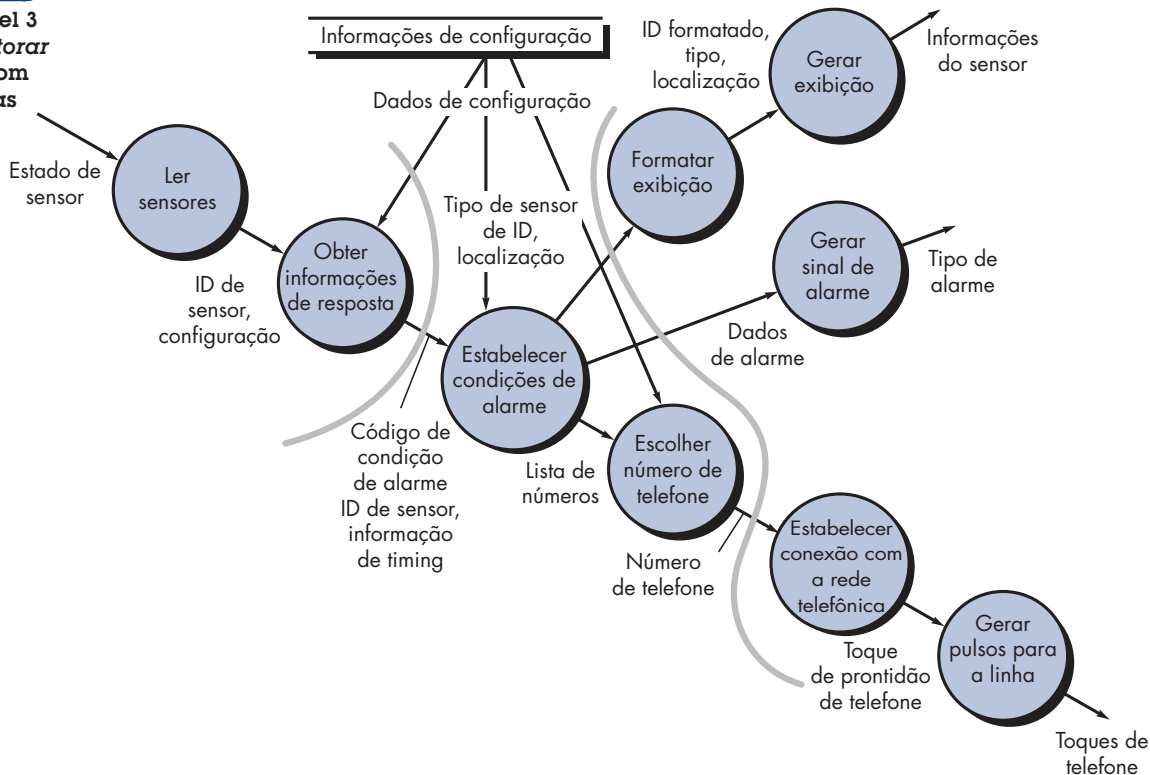
Passo 3. Determinar se o DFD possui características de fluxo de transformação ou transação¹¹. Avaliando-se o DFD (Figura 9.13), podemos ver dados entrando no software ao longo de uma trajetória de entrada e saindo ao longo de três trajetórias de saída. Consequentemente, uma característica de transformação geral será considerada para o fluxo de informações.

Passo 4. Isolar o centro de transformação especificando as fronteiras do fluxo de entrada e saída. Fluxos de dados de entrada ao longo de uma trajetória em que as informações são convertidas da forma externa para a forma interna; o fluxo que sai converte dados internalizados para a forma externa. As fronteiras do fluxo de entrada e saída são de livre interpretação. Ou seja, diferentes projetistas talvez escolham pontos ligeiramente diferentes do fluxo como pontos limítrofes. De fato, soluções de projeto alternativas podem ser obtidas variando-se o posicionamento das fronteiras do fluxo. Embora deva se tomar cuidado na escolha das fronteiras, uma variação de uma bolha ao longo de uma trajetória de fluxo em geral terá pouco impacto sobre a estrutura de programas final.

As fronteiras dos fluxos para o exemplo são ilustradas na forma de curvas cinza caminhando verticalmente através do fluxo na Figura 9.13. As transformações (bolhas) que constituem o centro de transformações estão entre os dois limites cinza que vão de cima para baixo na figura. Poderia se argumentar a necessidade de reajustar uma fronteira (por exemplo, poderia ser proposta uma fronteira do fluxo de entrada separando *ler sensores* e *obter informações de resposta*). A ênfase nessa etapa do projeto deve ser em selecionar fronteiras razoáveis, e não uma longa iteração para colocação de divisões.

FIGURA 9.13

DFD de nível 3 para monitorar sensores com as fronteiras do fluxo



¹¹ No fluxo de transações, um único dado, denominado *transação*, faz com que o fluxo de dados se ramifique ao longo de uma série de trajetórias de fluxo definida pela natureza da transação.

Passo 5. Realizar a “fatoração de primeiro-nível”. A arquitetura de programas obtida usando-se esse mapeamento resulta em uma distribuição do controle top-down. A *fatoração* conduz a uma estrutura de programas em que os componentes de alto nível tomam as decisões e os componentes de baixo nível realizam a maior parte do trabalho de entrada, cálculos e saída. Os componentes de nível médio realizam parte das tarefas de controle e volumes de trabalho moderados.

Ao se encontrar um fluxo de transformação, um DFD é mapeado em uma estrutura específica (uma arquitetura de chamadas e retornos) que fornece controle para o processamento de informações de entrada, saída e transformação. A fatoração de primeiro nível para o subsistema *monitorar sensores* é ilustrada na Figura 9.14. Um controlador principal (chamado *executivo para monitorar sensores*) reside na parte superior da estrutura de programas e coordena as seguintes funções de controle subordinadas:

- Um controlador de processamento de informações de entrada, denominado *controlador de entrada do sensor*, coordena o recebimento de todos os dados de entrada.
- Um controlador de transformação de fluxo, denominado *controlador de condições de alarme*, supervisiona todas as operações sobre os dados na forma internalizada (por exemplo, um módulo que chama vários procedimentos de transformação de dados).
- Um controlador de processamento de informações de saída, denominado *controlador de saída de alarme*, coordena a produção de informações de saída.

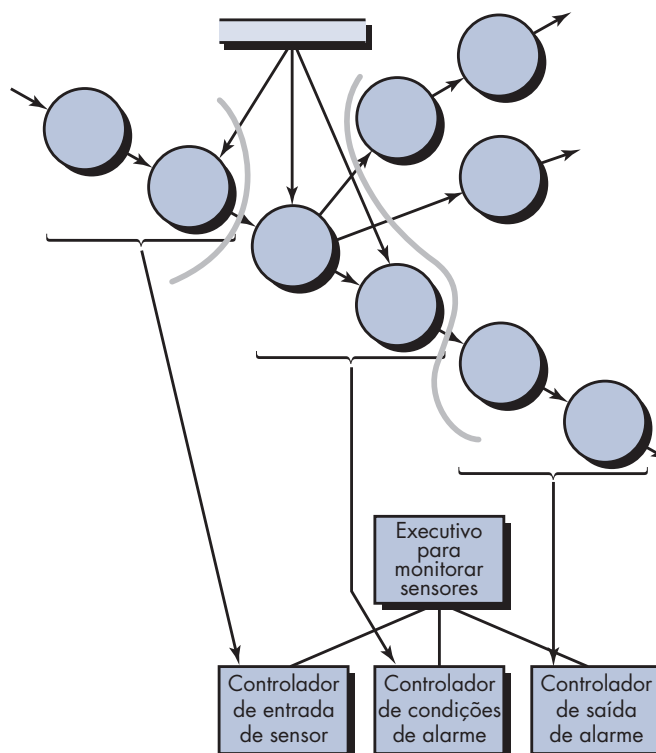
Embora pela Figura 9.14 esteja implícita uma estrutura de três flancos, fluxos complexos em sistemas grandes poderiam ditar dois ou mais módulos de controle para cada uma das funções de controle genéricas descritas anteriormente. O número de módulos no primeiro nível deve ser limitado ao mínimo capaz de realizar funções de controle e ainda assim manter boas características de independência funcional.



Não se torne dogmático neste estágio. Talvez seja necessário estabelecer dois ou mais controladores para cálculo ou processamento de entrada, baseando-se na complexidade do sistema a ser construído. Se o bom senso ditar esta abordagem, faça-a!

FIGURA 9.14

Fatoração de primeiro nível para monitorar sensores





Elimine módulos de controle redundantes. Ou seja, se um módulo de controle não fizer nada mais que controlar um outro módulo, sua função de controle poderia ser implodida para um módulo de nível mais alto.



Mantenha em nível baixo os módulos “trabalhadores” dentro da estrutura de programas. Isso levará a uma arquitetura mais fácil de ser mantida.

Passo 6. Realizar “fatoração de segundo nível”. A fatoração de segundo nível é realizada pelo mapeamento de transformações (bolhas) individuais de um DFD em módulos apropriados dentro da arquitetura. Partindo do limite do centro de transformação e movendo-se para a parte externa ao longo de trajetórias de entrada e depois de saída, as transformações são mapeadas em níveis subordinados da estrutura de software. A abordagem geral da fatoração de segundo nível é ilustrada na Figura 9.15.

Embora a Figura 9.15 ilustre um mapeamento um-para-um entre transformações DFD e módulos de software, frequentemente ocorrem mapeamentos diferentes. Duas ou até mesmo três bolhas podem ser combinadas e representadas como um componente ou uma única bolha poderia ser expandida em dois ou mais componentes. Considerações práticas, bem como medidas de qualidade do projeto, ditam o resultado da fatoração de segundo nível. Revisão e refinamento podem levar a mudanças nessa estrutura, porém pode servir como uma “primeira iteração” de projeto.

A fatoração de segundo nível para fluxo de entrada segue da mesma maneira. A fatoração é mais uma vez realizada movendo-se para fora a partir do limite do centro de transformação ao longo do fluxo de entrada. O centro de transformação do software de subsistema *monitorar sensores* é mapeado de maneira ligeiramente distinta. Cada conversão de dados ou transformação de cálculos da parte de transformação da DFD é mapeada em um módulo subordinado ao controlador de transformação. A Figura 9.16 mostra uma arquitetura de primeira iteração completa.

FIGURA 9.15
Fatoração de segundo nível para monitorar sensores

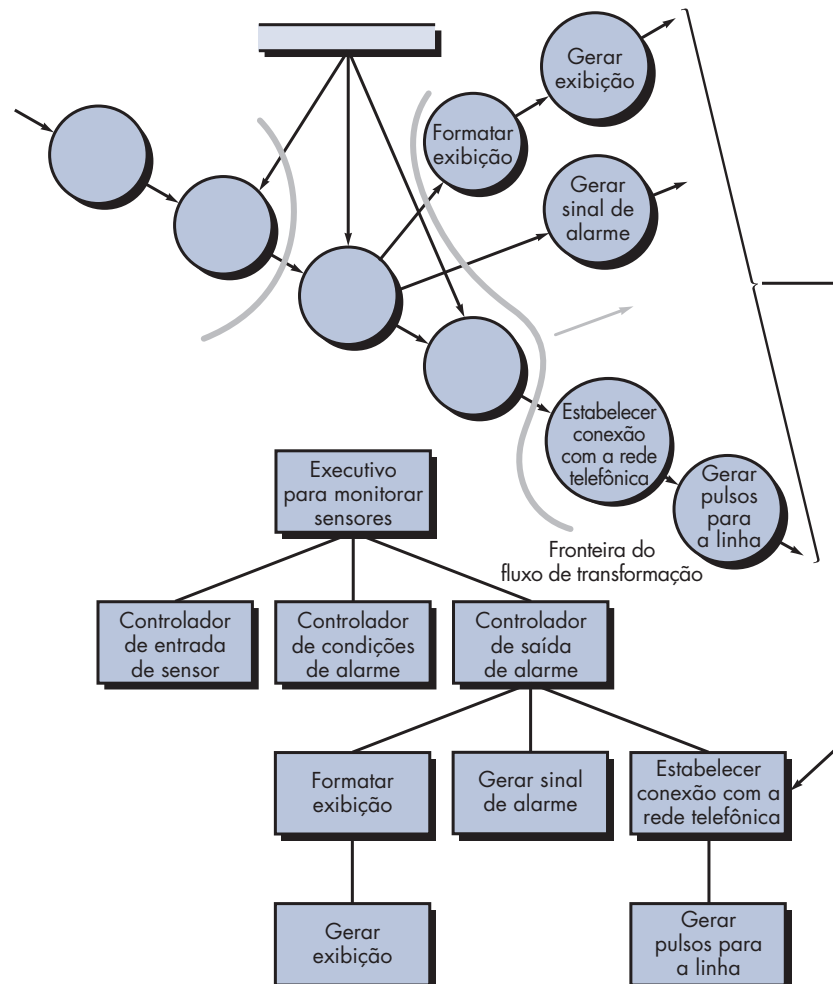
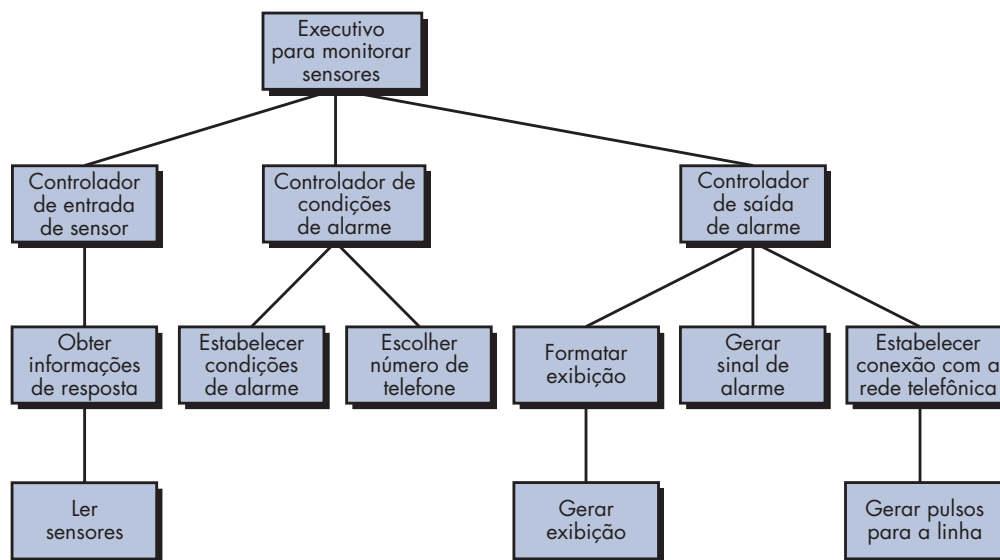


FIGURA 9.16

Estrutura da primeira iteração para monitorar sensores



Os componentes mapeados da maneira anterior e mostrados na Figura 9.16 representam um projeto inicial de arquitetura de software. Embora os componentes sejam chamados para que impliquem função, deve ser redigida uma breve narrativa de processamento (adaptada da especificação de processos desenvolvida para a transformação de dados criada durante a modelagem de requisitos) para cada um deles. A narrativa descreve a interface dos componentes, estruturas de dados internas, uma narrativa funcional e uma breve discussão das restrições e características especiais (por exemplo, entrada/saída de arquivos, características dependentes de hardware, requisitos de timing especiais).

“Faça as coisas da forma mais simples possível. Mas não mais simples do que isso.”

Albert Einstein

Passo 7. Refinar a primeira iteração de arquitetura usando heurística de projeto para aumentar a qualidade do software. Uma primeira iteração de arquitetura sempre pode ser refinada aplicando-se conceitos de independência funcional (Capítulo 8). Componentes são explodidos ou implodidos para produzir a sensata separação de preocupações, coesão adequada, acoplamento mínimo e, o mais importante, uma estrutura que possa ser implementada sem dificuldade, testada sem atribuições e mantida sem aflição.

Os refinamentos são ditados pela análise e métodos de avaliação descritos anteriormente na Seção 9.5, bem como por considerações práticas e bom senso. Há vezes em que, por exemplo, o controlador para fluxo de dados de entrada é completamente desnecessário, quando algum processamento de entrada é necessário em um componente subordinado ao controlador de transformação, quando o alto acoplamento devido a dados globais não pode ser evitado ou quando características estruturais ótimas não podem ser obtidas. Os requisitos de software associados ao discernimento humano são o árbitro final.

O objetivo das sete etapas anteriores é o de desenvolver uma representação de arquitetura de software. Uma vez definida a estrutura, podemos avaliar e refinar a arquitetura de software visualizando-a como um todo. Modificações feitas nessa ocasião requerem pouco trabalho adicional, embora possam ter um impacto profundo na qualidade do software.

Você deve fazer uma pausa por um momento e considerar a diferença entre a abordagem de projeto descrita e o processo de “escrever programas”. Se o código é a única representação de software, você e seus colegas terão grande dificuldade em avaliar ou refinar em um nível global ou holístico e terão, de fato, dificuldade “em distinguir a floresta das árvores”, ou seja, se você ficar muito envolvido com os detalhes não será capaz de ver os pontos importantes como um todo.

CASA SEGURA



Refinamento da arquitetura preliminar

Cena: Sala do Jamie, quando é iniciada a modelagem de projeto.

Atores: Jamie e Ed — membros da equipe de engenharia de software do *CasaSegura*.

Conversa:

[Ed acabou de completar um projeto preliminar do subsistema para monitorar sensores. Ele dá uma passada na sala para pedir a opinião do Jamie.]

Ed: Então, aqui está a arquitetura que obtive. [Ed mostra ao Jamie a Figura 9.16, que ele estuda por alguns instantes.]

Jamie: Excelente, mas acredito que possamos fazer algumas pequenas modificações para torná-la mais simples... E melhor.

Ed: Tais como?

Jamie: Bem, por que você usou o componente *controlador de entrada de sensor*?

Ed: Porque precisamos de um controlador para o mapeamento.

Jamie: Não necessariamente. O controlador não faz muita coisa, já que estamos controlando uma única trajetória de fluxos para os dados de entrada. Podemos eliminar o controlador sem provocar efeitos adversos.

Ed: Tudo bem, sem problemas. Farei a mudança e...

Jamie (sorrindo): Espere! Também podemos implodir os componentes *estabelecer condições de alarme* e *escolher número de telefone*. O controlador de transformações não é realmente necessário e a pequena diminuição na coesão é tolerável.

Ed: Simplificação, hein?

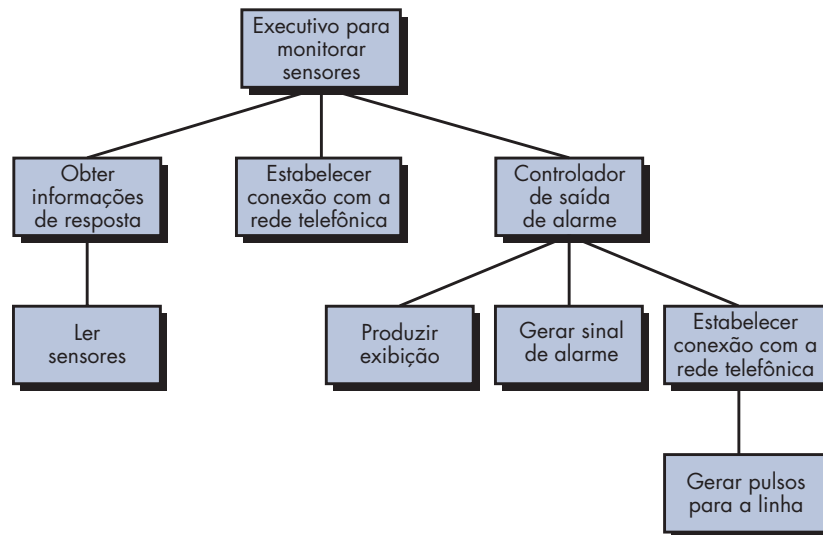
Jamie: Isso mesmo. E enquanto estivermos fazendo os refinamentos, seria uma boa ideia implodir os componentes *formatar exibição* e *gerar exibição*. A formatação de exibição para o painel de controle é simples. Podemos definir um novo módulo chamado *produzir exibição*.

Ed (rascunhando): Então é isso que você acha que deveríamos fazer? [Mostrando ao Jamie a Figura 9.17.]

Jamie: É um bom ponto de partida.

FIGURA 9.17

Estrutura de programas refinada para monitorar sensores



9.6.2 Refinamento do projeto da arquitetura

O que acontece após a arquitetura ter sido criada?

Qualquer discussão de refinamento de projeto deveria ser antecedida pelo seguinte comentário: “Lembre-se de que um ‘projeto ótimo’ que não funciona é de mérito questionável”. Devemos estar preocupados com o desenvolvimento de uma representação de software que atenderá todos os requisitos funcionais e de desempenho e mereça aceitação com base em medidas e heurísticas de projeto.

O refinamento da arquitetura de software durante os estágios iniciais do projeto deve ser encorajado. Conforme discutido neste capítulo, estilos de arquitetura alternativos podem ser obtidos, refinados e avaliados para a “melhor” abordagem. A abordagem para a otimização é um dos verdadeiros benefícios obtidos ao se desenvolver uma representação da arquitetura de software.

É importante notar que a simplicidade estrutural normalmente reflete tanto elegância quanto eficiência. O refinamento de projeto deveria se esforçar por obter o menor número de componentes que seja consistente com efetiva modularidade e a estrutura de dados menos complexa que atenda adequadamente os requisitos de informações.

9.7 RESUMO

A arquitetura de software fornece uma visão holística do sistema a ser construído. Ela representa a estrutura e a organização dos componentes de software, suas propriedades e as conexões entre eles. Os componentes de software incluem módulos de programas e as várias representações de dados manipuladas pelo programa. Consequentemente, o projeto de dados é parte integrante da obtenção da arquitetura de software. A arquitetura destaca decisões de projeto iniciais e fornece um mecanismo para considerar os benefícios de estruturas alternativas do sistema.

Uma série de estilos e padrões de arquitetura encontra-se à disposição do engenheiro de software e pode ser aplicada em um gênero de arquitetura. Cada estilo descreve uma categoria de sistema que engloba um conjunto de componentes que realiza uma função exigida por um sistema; um conjunto de conectores que possibilita a comunicação, coordenação e cooperação entre os componentes; restrições que definem como os componentes podem ser integrados para formar o sistema; e modelos semânticos que permitem a um projetista compreender as propriedades gerais de um sistema.

De modo geral, o projeto da arquitetura é realizado usando-se quatro etapas distintas. Primeiramente, o sistema deve ser representado no contexto. Ou seja, o projeto deve definir as entidades externas com as quais o software interage e a natureza da interação. Uma vez que o contexto tenha sido especificado, o projetista deve identificar um conjunto de abstrações de alto nível, denominado arquétipos, que representam elementos fundamentais do comportamento ou função do sistema. Depois de as abstrações definidas, o projeto começa a se aproximar do domínio da implementação. Os componentes são identificados e representados no contexto de uma arquitetura que os suporta. Por fim, são desenvolvidas instâncias específicas da arquitetura para “provar” o projeto em um contexto do mundo real.

Como um exemplo simples de projeto da arquitetura, o método de mapeamento apresentado neste capítulo usa características de fluxo de dados para obter um estilo de arquitetura comumente usado. Um diagrama de fluxo de dados (DFD) é mapeado em uma estrutura de programas usando uma abordagem de mapeamento de transformação. O mapeamento de transformação é aplicado a um fluxo de informações que apresenta fronteiras distintas entre dados de entrada e saída. O DFD é mapeado em uma estrutura que aloca controle para a entrada, o processamento e a saída ao longo de três hierarquias de módulos fatorados separadamente. Uma vez que a arquitetura tenha sido obtida, ela é elaborada e analisada usando-se critérios de qualidade.

PROBLEMAS E PONTOS A PONDERAR

- 9.1.** Usando a arquitetura de uma casa ou edifício como metáfora, faça comparações com arquitetura de software. Em que sentido as disciplinas da arquitetura clássica e arquitetura de software são similares? Como diferem?
- 9.2.** Apresente dois ou três exemplos de aplicações para cada um dos estilos de arquitetura citados na Seção 9.3.1.
- 9.3.** Alguns dos estilos de arquitetura citados na Seção 9.3.1 são hierárquicos por natureza e outros não. Faça uma lista de cada um dos tipos. Como os estilos de arquitetura que não são hierárquicos seriam implementados?
- 9.4.** Os termos *estilo de arquitetura*, *padrão de arquitetura* e *framework* (não discutido neste livro) são muitas vezes encontrados em discussões sobre arquitetura de software. Pesquise e descreva como cada um deles difere de seus equivalentes.

- 9.5.** Escolha uma aplicação com a qual esteja familiarizado. Responda cada uma das perguntas apresentadas para controle e dados na Seção 9.3.3.
- 9.6.** Pesquise o ATAM (usando [Kaz98]) e apresente uma discussão detalhada das seis etapas indicadas na Seção 9.5.1.
- 9.7.** Caso já não tenha feito, complete o Problema 6.6. Use os métodos de projeto descritos no presente capítulo para desenvolver uma arquitetura de software para a PHTRS.
- 9.8.** Usando um diagrama de fluxo de dados e uma narrativa de processamento, descreva um sistema baseado em computador que tenha características distintas de fluxo de transformações. Defina as fronteiras dos fluxos e mapeie o DFD em uma arquitetura de software usando a técnica descrita na Seção 9.6.1.

LEITURAS E FONTES DE INFORMAÇÃO COMPLEMENTARES

A literatura sobre arquitetura de software explodiu ao longo da última década. Livros como os de Gorton (*Essential Software Architecture*, Springer, 2006), Reekie e McAdam (*A Software Architecture Primer*, Angophora Press, 2006), Albin (*The Art of Software Architecture*, Wiley, 2003) e Bass e seus colegas (*Software Architecture in Practice*, 2. ed., Addison-Wesley, 2002) apresentam úteis introduções para uma área intelectualmente desafiadora.

Buschman e seus colegas (*Pattern-Oriented Software Architecture*, Wiley, 2007) e Kuchana (*Software Architecture Design Patterns in Java*, Auerbach, 2004) discutem aspectos orientados a padrões de projeto da arquitetura. Rozanski e Woods (*Software Systems Architecture*, Addison-Wesley, 2005), Fowler (*Patterns of Enterprise Application Architecture*, Addison-Wesley, 2003), Clements e seus colegas (*Documenting Software Architecture: View and Beyond*, Addison-Wesley, 2002), Bosch [Bos00] e Hofmeister e seus colegas [Hof00] oferecem completos tratados sobre arquitetura de software.

Hennessey e Patterson (*Computer Architecture*, 4. ed., Morgan-Kaufmann, 2007) adotam uma visão quantitativa distinta sobre as questões de projeto da arquitetura de software. Clements e seus colegas (*Evaluating Software Architectures*, Addison-Wesley, 2002) consideram as questões associadas com a avaliação de alternativas de arquitetura e a escolha da melhor arquitetura para determinado domínio de problema.

Livros abordando implementações específicas sobre arquitetura tratam o projeto da arquitetura em uma tecnologia ou ambiente de desenvolvimento específico. Marks e Bell (*Service-Oriented Architecture*, Wiley, 2006) discutem um método de projeto que associa recursos comerciais e computacionais aos requisitos definidos por clientes. Stahl e seus colegas (*Model-Driven Software Development*, Wiley, 2006) discutem a arquitetura no contexto de abordagens de modelagem específicas a um domínio de aplicação. Radaideh e Al-ameed (*Architecture of Reliable Web Applications Software*, GI Global, 2007) consideram arquiteturas apropriadas para WebApps. Clements e Northrop (*Software Product Lines: Practices and Patterns*, Addison-Wesley, 2001) tratam o projeto de arquiteturas que suportam linhas de produtos de software. Shanley (*Protected Mode Software Architecture*, Addison-Wesley, 1996) fornece uma orientação para projeto da arquitetura para qualquer um que esteja desenvolvendo sistemas operacionais em tempo real baseados em PCs, sistemas operacionais multi-tarefa ou drivers de dispositivos.

A pesquisa atual em arquitetura de software é documentada anualmente nos *Proceedings of the International Workshop on Software Architecture*, patrocinados pela ACM e outras organizações de computação e nos *Proceedings of the International Conference on Software Engineering*.

Uma ampla gama de fontes de informação sobre projeto da arquitetura se encontra à disposição na Internet. Uma lista atualizada de referências na Web, relevante para o projeto da arquitetura, pode ser encontrada no site www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm.